



University of Pisa

Computer Engineering & Artificial Intelligence and Data Engineering

Large Scale and Multi-structured databases

Nexus Scholar

Group members:

Andrea Covelli

Daniele Congiusti

Luca Giannini

Academic Year 2025/2026

Contents

1	Introduction	4
1.0.1	GitHub Repository	4
1.1	Researchers world exploration	4
1.2	Powerful analytics	4
2	Actors and Requirements	5
2.1	Actors	5
2.2	Requirements	6
2.3	Functional Requirements	6
2.3.1	Unregistered User	6
2.3.2	Registered User	6
2.3.3	Admin	6
2.3.4	Authors	6
2.3.5	Paper	6
2.4	Non-Functional Requirements	7
2.5	UML Class diagram	8
3	User guide	9
3.1	Manual	9
3.1.1	Homepage	9
3.1.2	Search page	9
3.1.3	Paper detail page	9
3.1.4	Author detail page	9
3.1.5	Analytics page	9
3.1.6	User page	9
3.1.7	Admin page	10
3.2	Mockups	11
4	Datasets and Models	17
4.1	Datasets	17
4.2	Models	19
4.2.1	Raw data	19
4.2.2	MongoDB	20
4.2.3	Neo4j	22
5	Implementation	26
5.1	config	26
5.2	security	26
5.3	controller	26
5.4	model.mongo	27
5.5	model.neo4j	29
5.6	dao	29
5.7	dao.mongo	30
5.8	dao.neo4j	30
5.9	Utils classes	30
5.10	dao.exception	31
5.11	dto.auth	31

5.12	dto.mongo	31
5.13	dto.neo4j	33
5.14	service	34
5.15	service.exception	35
5.16	service.impl	35
6	Relevant Queries	37
6.1	MongoDB	37
6.1.1	Venue Impact Analysis	37
6.1.2	Collaboration Evolution Analysis	38
6.1.3	Trend Analysis and Hot Topics	40
6.1.4	Prolific Author Identification	41
6.1.5	PaperLeaderboard	43
6.2	Neo4j	45
6.2.1	Page ranking papers	45
6.2.2	Shortest Collaboration Path	46
6.2.3	Hidden Leiden Communities	48
6.2.4	Betweenness of papers	49
7	Design Considerations	52
7.1	Indexes	52
7.1.1	Full-Text Search Implementation (Smart Search)	52
7.1.2	Optimization of Prefix Search (Autocomplete)	53
7.1.3	Neo4j Author name index	54
7.2	Sharding	55
7.2.1	Sharding Strategy	55
7.3	Usage of Virtual Machines	55
7.4	CAP Theorem reflections	56
7.4.1	Neo4j	56
7.4.2	MongoDB	56
7.5	Management of the consistency between the two types of Databases	56
8	Testing and Validation	57
8.1	Testing Stack	57
8.2	Test Categories	57
8.2.1	Unit Tests	57
8.2.2	Integration Tests	57
8.3	Continuous Integration (CI)	58
9	Conclusions	59
9.1	Future works	59

Chapter 1

Introduction

1.0.1 GitHub Repository

Link to GitHub repository: <https://github.com/AndreaCovelli/NexusScholar>

1.1 Researchers world exploration

Nexus Scholar is an application provided for users who are interested in joining into the world of researchers, full of publications and links in various topics like Computer Science, Physics and so on. Nowadays, a lot of people are involved in writing papers alone or doing collaboration with several researchers and in fact these papers are useful for other papers, that cites previous one for doing some conclusion or calculation. It is essential to discover a complete knowledge of a topic, the exploration of correlated papers and retrieve some useful information about authors, operation not simple at all due to the large amount of authors and papers that travels in several educational platforms. Here it comes the usage of our application, we offer to users the possibility to retrieve information about papers, authors and topics, adding also some analytics that could be useful to observe important result as the collaboration path between two authors or the trend of a particular Topic, and so on.

More explanation of this "world" it is necessary to explain something else: Papers can be written by one or more Authors and can also cite one or more papers, this is used to conclude two things, papers create a collaboration between authors and papers are connected among themselves thanks to citations. Another aspect to consider is that papers describe a micro topic, but this one is grouped with other micro topics in a single macro topic (or simply topic), this fact is used to obviously classify papers and discover what is the field of study of single authors.

1.2 Powerful analytics

Here it follows main features that the application will offer our users:

- **Influence of papers:** Discover researchers and most influential papers using ranking algorithms (i.e. PageRank)
- **Link authors:** Figure out shortest collaboration paths between two researchers (Erdős number)
- **Evolving trends :** research emerging trends over time by calculating the number of papers published for each year of every field of study, making possible also to compare different topics
- **Paper bookmark :** Let user to bookmark papers that maybe will be read in a future time
- **More about authors :** Discover information and statistics about authors, like number of papers published, topics he is involved in, and so on.
- **Leiden communities :** Using the Leiden algorithm, it is possible to classify authors in various communities,

Chapter 2

Actors and Requirements

2.1 Actors

In our application we manage three kinds of actors:

1. **Unregistered User** : A user who simply decides to access. access the application without any kind of registration, typically done by casual users
 - Research papers by title or by authors who have written it.
 - Explore Authors' profiles with limited amount of information such as restricted analyses about collaboration path, publication summaries and so on
 - Analyses offered are restricted: only a few results are shown
2. **Registered User** : User that has done registration using the application and use Nexus Scholar frequently
 - All privileges guaranteed to unregistered user are also available
 - Visualize the citation network, where we can connect papers between themselves by the citation method
 - Complex Analyses are available, like the Collaboration Evolution and the Trend Analyses, more details in chapter 6.
 - Save papers that the user has read using the bookmark feature, where the user stores papers they are interested in and can retrieve quickly accessing in bookmarked papers section
3. **Admin** : Person who has the responsibility to maintain the application in a consistent state and supervise the behavior of users and state of database
 - Managing users considering ban when users don't respect policies
 - Monitor the entire database in order to maintain consistency and to avoid critical situation that will influence the performance of algorithms (for example Papers without authors)

2.2 Requirements

2.3 Functional Requirements

2.3.1 Unregistered User

ID	Description
FR-UU-01	An Unregistered User can become a Registered User through registration and login.
FR-UU-02	An Unregistered User can do basics analytics and searches.

2.3.2 Registered User

ID	Description
FR-RU-01	A Registered User can do all the analytics and search of the unregistered user
FR-RU-02	A Registered User can do all the complex queries and analytics.
FR-RU-03	A Registered User can add a paper to his bookmarked papers.
FR-RU-04	A Registered User can do CRUD operations on papers and authors to the databases.
FR-RU-05	The username of the Registered User must be unique.
FR-RU-06	The mail of the Registered User must be unique.
FR-RU-07	His bookmarked papers cannot contain duplicates.
FR-RU-08	When a Registered User add a paper to his bookmarked papers, the system must control that the paper exists in the database.

2.3.3 Admin

ID	Description
FR-AD-01	Administrators shall be able to ban registered users who break the system's contract rules.
FR-AD-02	The username of the Administrator must be unique.
FR-AD-03	The mail of the Administrators must be unique.

2.3.4 Authors

ID	Description
FR-PP-01	The S2 ID must be unique.
FR-PP-02	All of the authors of a paper must exist in the database.
FR-PP-03	A paper needs to have at least one author to exist in the database

2.3.5 Paper

ID	Description
FR-AU-01	The doi of the paper must be unique.
FR-AU-02	To link a paper to an author's history, the referenced paper must already exist in the database.
FR-AU-03	A paper needs to have at least one author to exist in the database

2.4 Non-Functional Requirements

Category	Requirement Description
Security	All passwords must be encrypted (e.g., using BCrypt) before being stored in the database.
Security	API endpoints must be stateless and secured via JWT tokens passed in the HTTP Authorization header.
Performance	Some particular searches should be optimized (using Indexes) to handle large datasets.
Interoperability	The system shall expose a RESTful API returning data in JSON format.
Maintainability	The codebase shall follow a layered architecture (MVC + DAO + DTO) to ensure separation of concerns.
CAP Theorem	The system must be CP to ensure data consistency and partition tolerance.
Platform Agnostic	The system must be deployed on the Docker Containers to ensure maximum compatibility with all the Operating System.
Robustness	The Code Coverage of the systems must be at least 90%.

2.5 UML Class diagram

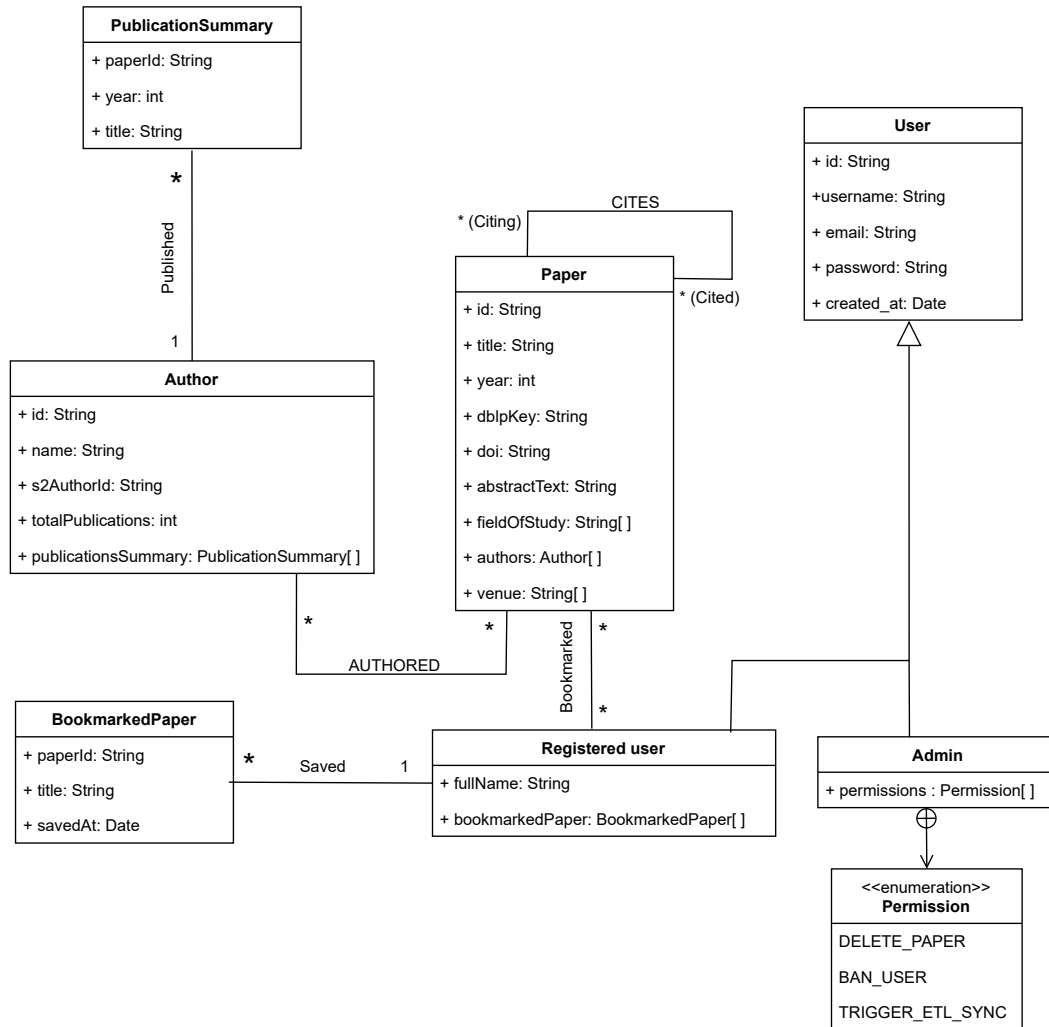


Figure 2.1: UML Class Diagram

Chapter 3

User guide

3.1 Manual

3.1.1 Homepage

In this page, the user receives a presentation of the application, where he could also click on *Sign In* to login or *Sign Up* to register. Authentication is necessary if user wants to enjoy all features of NexusScholar, but it could also start exploring what our application can offer with limits. A reference to the previous one is at Figure 3.1

3.1.2 Search page

In order to permit the user to explore contents we offer a search method that allows the user to insert the title or keywords or author of a specific paper, then results that are coherent with the query will be displayed below the bar, like in the Figure 3.2. As it could be seen several papers will be shown and user can click on the title or one of correlated links, to figure out more details about selected paper.

3.1.3 Paper detail page

Once user has chosen to see more details about a paper, a page like the one in Figure 3.3 is represented. Here we can observe the content of the paper and in addition, its citations and some analytics computed with specific algorithms (like pagerank or co-author graph computations). If the user is interested about the paper shown, he can also save this paper (bookmark), permitting to read in the future accessing his bookmark section.

3.1.4 Author detail page

If someone is interested in a particular author that has written paper we are interested in, there is the possibility to figure out more details on the author detail page, one example of the preceding is in the Figure 3.4. Where we can see small registry about the author and below one panel with publications that he or she has done and another panel where we can observe a collaboration pathway panel, where we can discover authors he has cooperated with.

3.1.5 Analytics page

Another page that is so useful to discover worthwhile information about the world of researchers is the Analytics page where user can choose which statistics want to compute and represent with charts. In the Figure 3.5 it could be seen a chart derived from the computation of hot topics analysis and correlated numbers as total publications, citations link and so on.

3.1.6 User page

In this page the user, once logged or registered, can see his personal informatio and there is also a panel where user can retrieve his bookmarked papers or delete it from the list like in the Figure 3.6. Another

feature provided to user is the possibility to execute analytics from his profile and obtain result without any kind of limits, like in the Figure 3.7

3.1.7 Admin page

Administrators have the role to manage the state of the server and moderate user community. To help admin doing so, we provide the *Admin page* that provides three main sections:

1. System overview: gives infos about the state of the server using a log and some results that are shown in the Figure 3.8 like number of papers, active users and so on.
2. User management: gives a perspective of all users registered with the possibility to ban them if necessary, admin can do so clicking the button at right of the specific user and then confirm the ban if all controls are done.
3. Content Moderation: permits the admin to remove a paper from the application specifying the title or the DOI, once spotted the correct paper, admin has to press the Delete button like in the Figure 3.10 .

3.2 Mockups

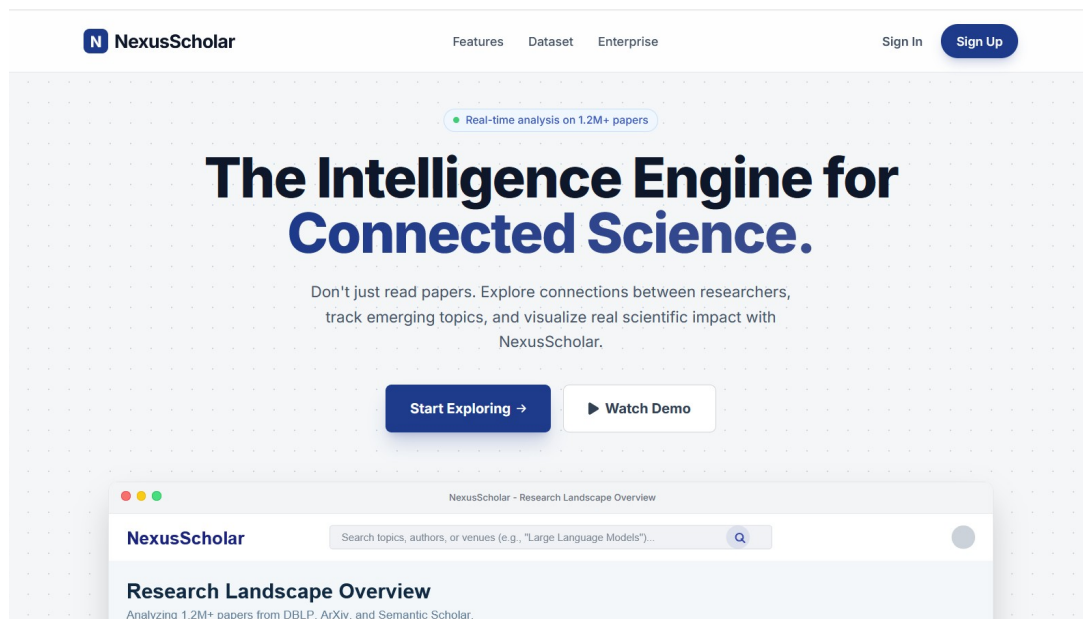


Figure 3.1: Home page NexusScholar

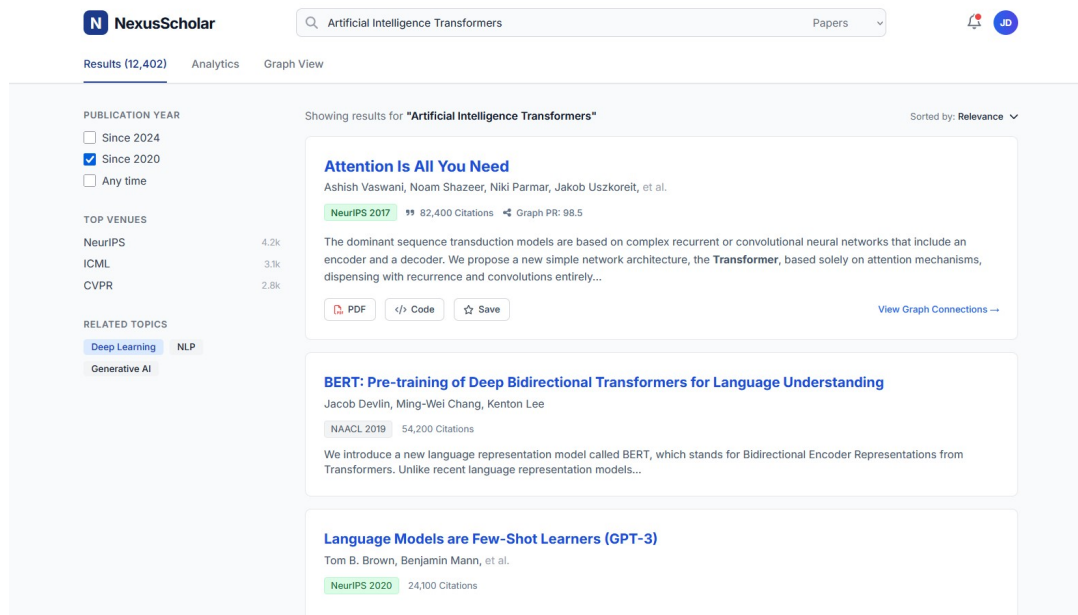


Figure 3.2: Search page

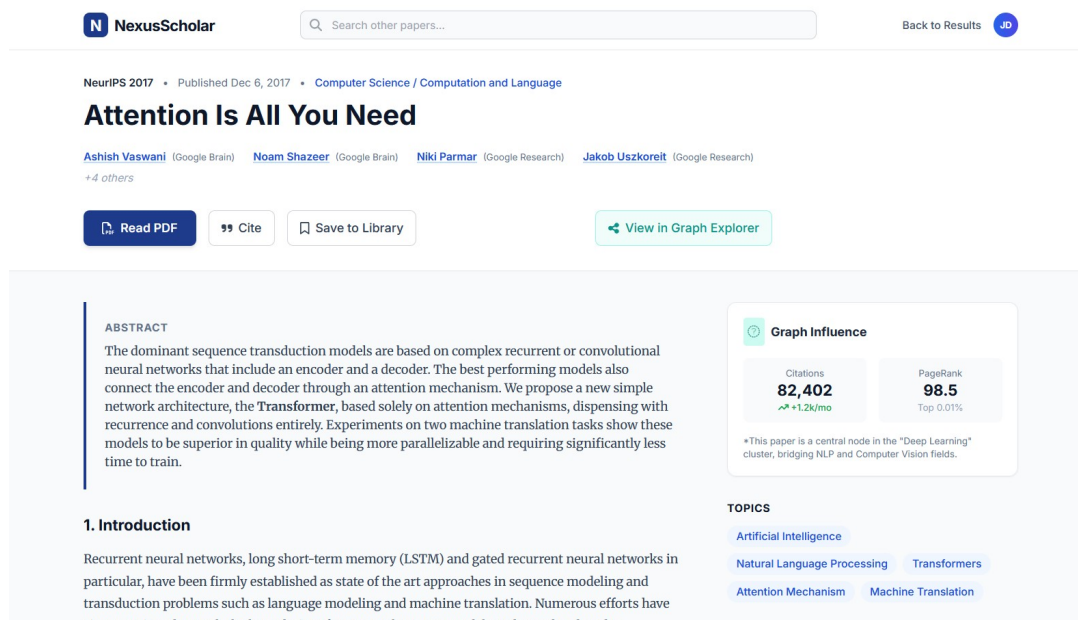


Figure 3.3: Paper detailed page

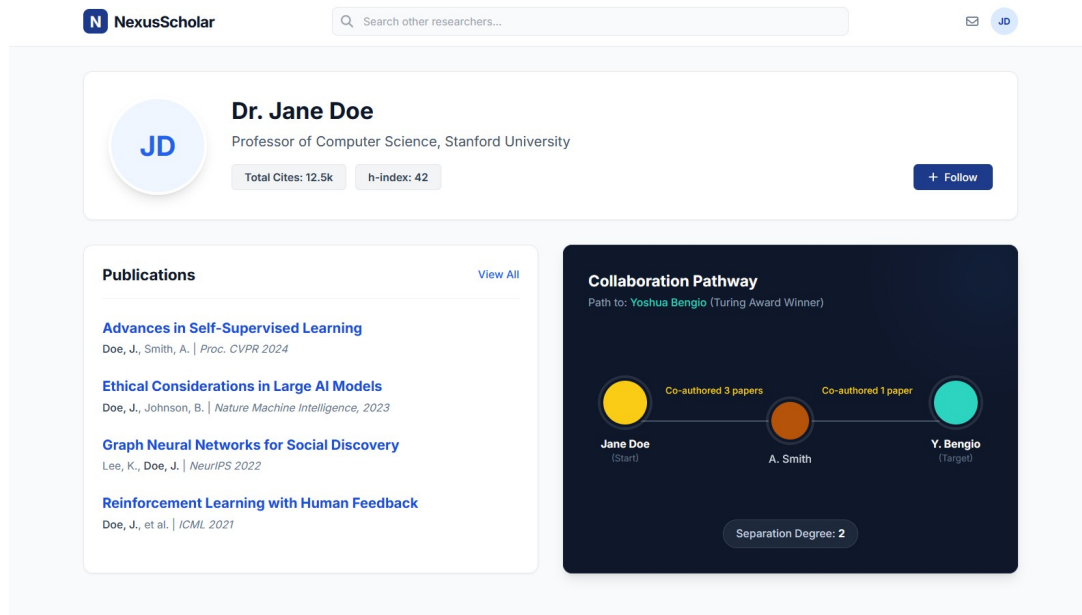


Figure 3.4: Detailed page of author

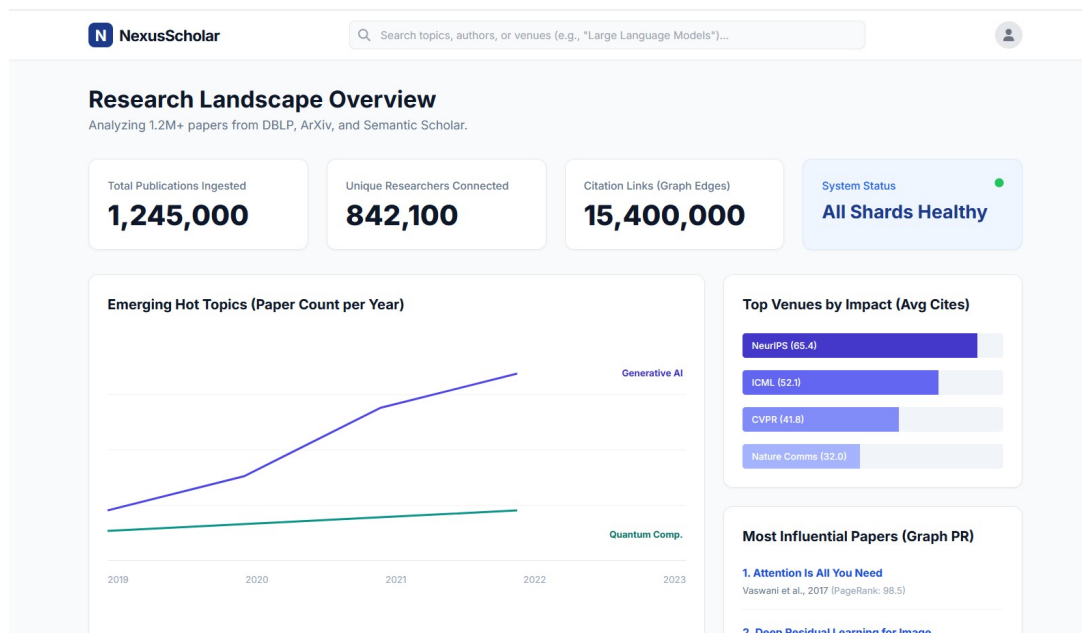


Figure 3.5: Example of analytics offered

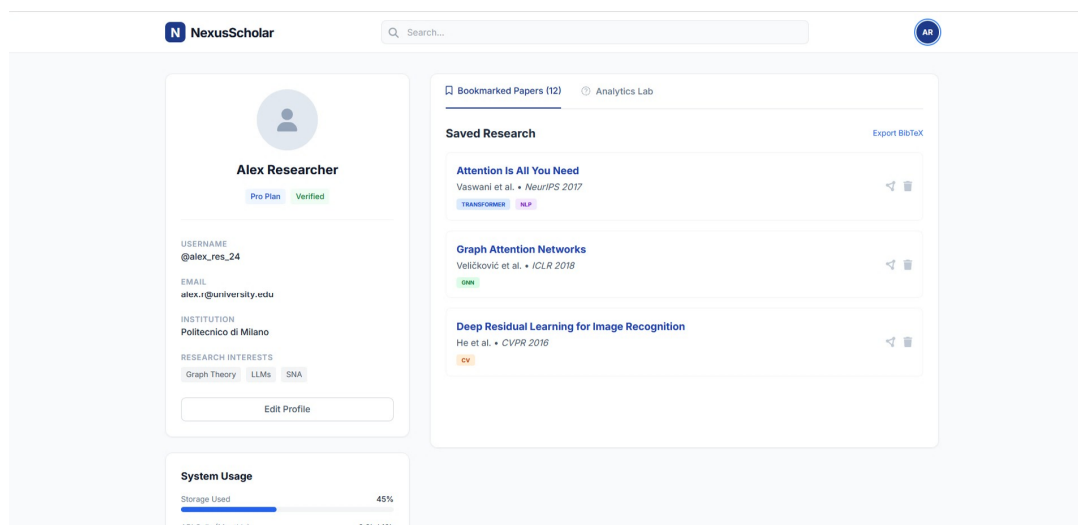


Figure 3.6: User personal bookmark area

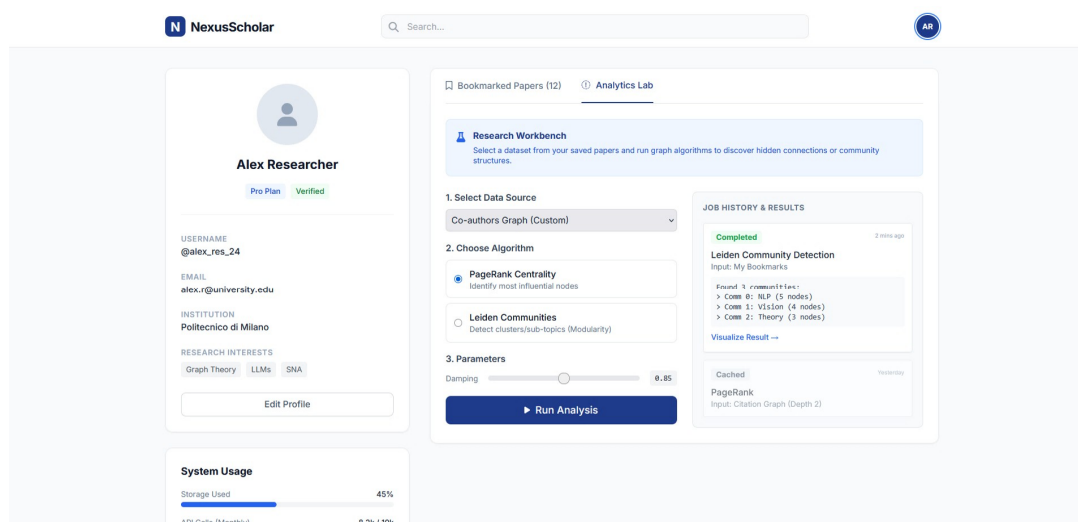


Figure 3.7: User personal Analytics Area

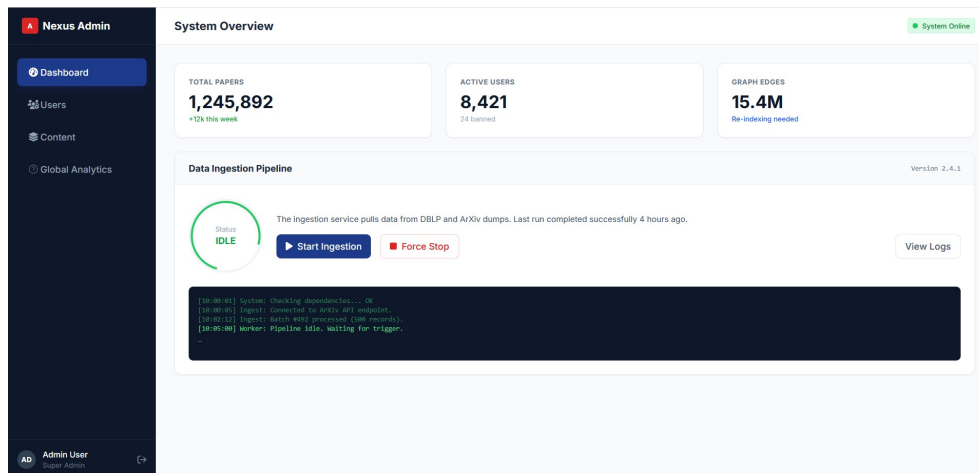


Figure 3.8: Status of system view

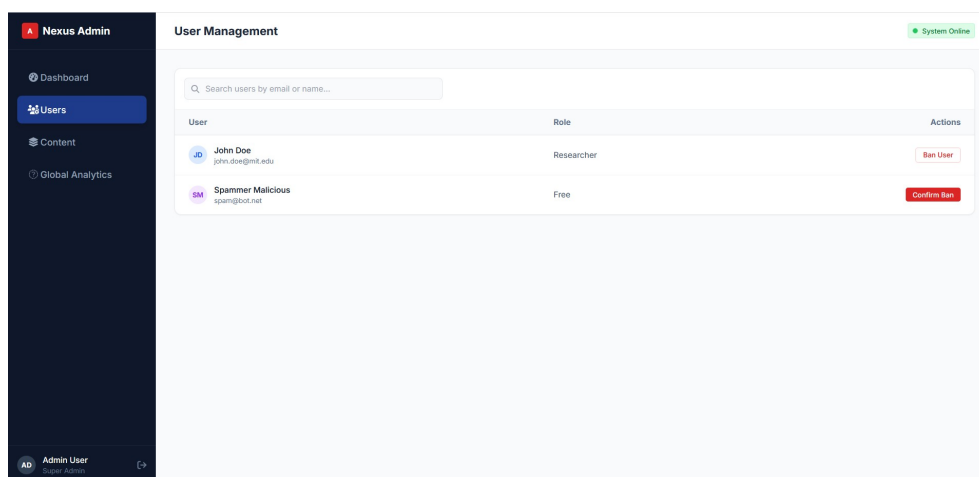


Figure 3.9: Example of analytics offered

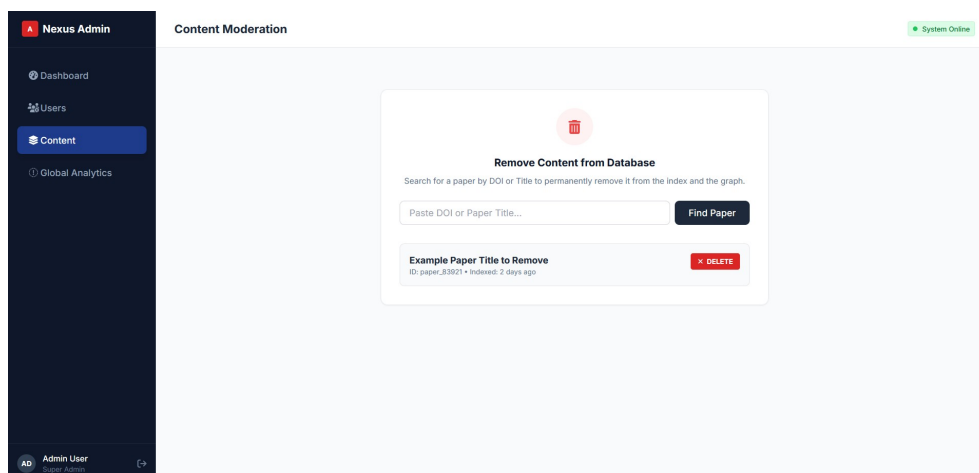


Figure 3.10: Management papers page

Chapter 4

Datasets and Models

4.1 Datasets

Data for Nexus Scholar application are retrieved from two sources **DBLP** and **Semantic Scholar API**, where the first one consists of a Computer Science Bibliography that contains main info about papers, authors and topics, and Semantic Scholar offers to us pre-print access across scientific fields. Before analyzing data-pipeline process, it is necessary to specify that these two sources provides data in 2 different formats: DBLP gives us XML Dump data, Semantic Scholar in JSON format instead. Before continuing discussion of datasets, let's focus on 3 Vs that characterize our data:

1. **Volume:** We're treating with high volumes of data, consider that only DBLP contains almost 1GB of data composed of more than 6 million papers and more than 3 million authors
2. **Variety:** It's medium-High due to data from 2 different sources that has also different formats, more specifically we got a XML Dump and a JSON API data that compose our dataset.
3. **Velocity/Variability:** Low-Medium, because DBLP updates its data monthly, so information doesn't change regularly, Semantic scholar represents opposite case instead, because updates are daily.



(a) DBLP logo



(b) Semantic Scholar Logo

To retrieve data correctly we provided some scripts in python, that will be explained subsequently to discuss the data-pipeline topic. Data-pipeline is composed by several phases where one script is executed, in detail we have 4 phases to load our data. Before starting the explanation of data-pipeline, in fact there are several steps to take:

1. Download **dblp.xml.gz** and **dblp.dtd** from dblp.org
2. Prepare the *API_KEY*¹ and credentials for access into databases
3. Generate a mongo key file in order to permit authentication in MongoDB replica set (this is done using specific commands)
4. Activate following Docker containers: 3 for mongoDB instances, 1 for Neo4j and then 1 for backend container, that responds at port 8080

Another step necessary to complete this phase is to create a python virtual environment, in order to add components (using pip) and execute scripts provided. Once all above steps are completed successfully effective data-pipeline phase could start:

¹Semantic Scholar API KEY can be requested at the site of Semantic Scholar site

1. **Parse DBLP Data:** in this phase will be created *dblp_ai_ml_2015_2025.jsonl* file with data of papers from dblp files downloaded before, more specifically the papers from last 10 years about Computer science topics. In this way we've retrieved a right amount of rows of data and formatted in a jsonl format, useful for successive steps.
2. **Enrichment (Semantic Scholar):** By doing API request we enrich our jsonl data file with additional information: abstracts, citations, and topics. All these results will be stored in *enrichment_cache.db* file, useful for next step.
3. **Integration & Formatting:** Once obtained these 2 files there is the need to arrange data into files to give as input of import commands for MongoDB and Neo4j instances. To do this we arrange data in order to resolve citation graphs, assign IDs to each found document, considering Authors (AXXXXXX), Papers (PXXXXXX), create users(both admins and registeredUsers), obtained using json files generated² to dispose users with username and email (remaining work will be done by script). Most important phase consists of formatting all these data collected, enriched and completed, leading us to have several .jsonl and csv files, used in final phase of this process.
4. **Data Loading:** In this phase, bulk imports are done to load data into docker containers for our database instances and, to make this possible, preceding phase's files must be created successfully, because a bash script will execute *mongoimport* and *neo4j-admin database import* commands, shown below.
5. **Index Creation:** In this final phase, we create the indexes into Mongo DB. More details in appropriate section.

```

1 # Define the connection string
2 MONGO_CONN="--uri=mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongo1:27017,mongo2
   :27017,mongo3:27017/${DB_NAME}?replicaSet=rs0&authSource=admin&w=majority"
3
4 echo "    Importing Authors..."
5 # Mount the local MongoDB import dir to /import inside the temporary container
6 # Use --drop to clear the collection before importing.
7 docker compose -f $COMPOSE_FILE run --rm -v $MONGO_IMPORT_DIR:/import mongo1
   mongoimport $MONGO_CONN --collection authors --file /import/authors.jsonl --
   drop
8
9 echo "    Importing Papers (This might take a while)..."
10 docker compose -f $COMPOSE_FILE run --rm -v $MONGO_IMPORT_DIR:/import mongo1
   mongoimport $MONGO_CONN --collection papers --file /import/papers.jsonl --drop
11
12 #Importing registered User
13 echo "    Importing registered Users..."
14 docker compose -f $COMPOSE_FILE run --rm -v $MONGO_IMPORT_DIR:/import mongo1
   mongoimport $MONGO_CONN --collection registeredUsers --file /import/
   registeredUsers.jsonl --drop
15
16 #Importing admin
17 echo "    Importing Admins..."
18 docker compose -f $COMPOSE_FILE run --rm -v $MONGO_IMPORT_DIR:/import mongo1
   mongoimport $MONGO_CONN --collection admins --file /import/admins.jsonl --drop

```

Listing 4.1: Mongoimport calls in load.data script

```

1 docker compose -f $COMPOSE_FILE run --rm --user neo4j neo4j neo4j-admin database
   import full \
2   --overwrite-destination=true \
3   --nodes=Paper=$NEO4J_IMPORT_DIR/nodes_papers.csv \
4   --nodes=Author=$NEO4J_IMPORT_DIR/nodes_authors.csv \
5   --nodes=Topic=$NEO4J_IMPORT_DIR/nodes_topics.csv \
6   --relationships=AUTHORED=$NEO4J_IMPORT_DIR/rels_authored.csv \
7   --relationships=CITES=$NEO4J_IMPORT_DIR/rels_cites.csv \
8   --relationships=HAS_TOPIC=$NEO4J_IMPORT_DIR/rels_has_topic.csv

```

Listing 4.2: Neo4j import calls in load.data script

²Using generatedata site

4.2 Models

In order to discuss models concepts, we'll start discussing the format of raw data, that comes from our source DBLP, Semantic Scholar and generatedata. We remark that we retrieve data from 2 different format, that will be explained below

4.2.1 Raw data

Let's start discussing DBLP format, that consists of an XML format where each paper considering:

- title
- number of pages
- publication year
- booktitle
- ee (Electronic Edition)
- url
- author

An example exposed in the following listing:

```
1 <dblp>
2 <incollection mdate="2017-07-12"
3   key="reference/vision/X14bd" publtype="encyclopedia">
4 <title>Curvedness.</title>
5 <pages>159</pages>
6 <year>2014</year>
7 <booktitle>Computer Vision, A Reference Guide</booktitle>
8 <ee>https://doi.org/10.1007/978-0-387-31439-6_100117</ee>
9 <url>db/reference/vision/vision2014.html#X14bd</url>
10 </incollection>
11 <incollection mdate="2019-01-09"
12   key="reference/vision/Singh14" publtype="encyclopedia">
13 <author>Manish Singh 0001</author>
14 </dblp>
```

On the other hand, we've to deal with json format that comes from Semantic Scholar API and an example of response given at url request <https://api.semanticscholar.org/graph/v1/paper/batch> is reported in the following listing:

```
1 {
2   "dblp_key": "conf/aaai/0001000FZ021",
3   "doi": "10.1609/aaai.v35i11.17231",
4   "status": "SUCCESS",
5   "s2_data": {
6     "paperId": "4e40105fc2b8dba251ed8c6239436e6dd12c3f78",
7     "title": "Unified Tensor Framework for Incomplete Multi-view Clustering and Missing-
8       view Inferring",
9     "referenceCount": 56,
10    "citationCount": 140,
11    "openAccessPdf": {
12      "url": "https://ojs.aaai.org/index.php/AAAI/article/download/17231/17038",
13      "status": "GOLD",
14      "license": null,
15      "disclaimer": "Notice..."
16    },
17    "fieldsOfStudy": [
18      "Computer Science"
19    ],
20    "authors": [
21      {
22        "authorId": "2113341206",
23        "name": "Jie Wen"
```

```

23     }
24   ],
25   "citations": [
26     { "paperId": "c3f349eb18c5346442f592725599d46a2531dd38" },
27     { "paperId": "23f3f4a04a14b7da38e99df34cef866c10f1b9b9" }
28   ],
29   "abstract": "In this paper, we propose a novel method, ..."
30 }
31 }

```

One detail to focus on is that in both types of data we encounter similar fields, like title, author and so on. These fields are maintained and used to make association between results from two different sources (Step 3 of data-pipeline), in order to unify data and create jsonl and csv files. Other words to spend are for the users raw data, that are obtained using generatedata site (mentioned before) , composed of two fields per user *name* and *email*. Previous data are stored in two json files (one for regular RegisteredUsers and one for admins), for complete the explanation of raw data it will be showed an example:

```

1 [
2   {
3     "name": "Norman McGee",
4     "email": "mollis@outlook.ca"
5   }
6 ]

```

We generated a total of 100 registered users and 10 admin reporting these 2 fields, that will be used by a python script in order to create jsonl files. Now we can move on explanation of the structure of files that will be input of import commands.

4.2.2 MongoDB

For loading data in our MongoDB instances we dispose 4 different jsonl files, respectively one for each collection:

1. Papers
2. Authors
3. Admins
4. Registered users

Each files respects specific format, let's start describing files derived from users raw data content: we've to say that 2 files are generated, one for Administrators and one for registered users, couple of documents are reported next:

```

1
2 *****
3 *   Admins.jsonl type document   *
4 *****
5
6 {
7   "_id": "AD01",
8   "username": "norman_mcgee_admin",
9   "email": "mollis@outlook.ca",
10  "password_hash": "$2b$12$1q2xuXN08zrtbdqVcqwe.7jwq63Y2ukET0Y9mYUjGgc9FSNzRr0y",
11  "created_at": "2025-09-03T19:50:46Z",
12  "permissions": [
13    "TRIGGER_ETL_SYNC",
14    "DELETE_PAPER",
15    "BAN_USER"
16  ]
17 }
18
19
20 *****
21 *   registeredUsers.jsonl type document   *
22 *****
23
24 {
25   "_id": "U000001",

```

```

26 "username": "nexusscholar_sigourney_landry",
27 "email": "eu.erat@aol.net",
28 "password_hash": "$2b$12$.IMtCcozKaolQzXaeYHyDOF7Le6C0cf20blsFxxKemmtvZa9zFLu",
29 "created_at": "2025-09-28T17:01:41Z",
30 "full_name": "Sigourney Landry",
31 "bookmarked_papers": [
32   {
33     "paper_id": "P000101",
34     "title": "Generalization Analysis for Game-Theoretic Machine Learning.",
35     "saved_at": "2026-02-09T17:01:41Z"
36   },
37   {
38     "paper_id": "P000574",
39     "title": "Near-Optimal Active Learning of...",
40     "saved_at": "2025-12-07T17:01:41Z"
41   }
42 ]
43 }

```

We obviously notice that formats has some common parts, like `username`, `password_hash`³ and so on. But we can also deduce some differences from these two kind of users data:

- Admin has a `permissions` field that contains allowed operations dedicated to manage system.
- Registered users contains `full_name` field and `bookmarked_papers` field, that contains the string with full name and a list of bookmarked papers informations⁴

Let's move now on entries of author files, as like before it is shown an example of entry:

```

1 {
2   "_id": "A00001",
3   "name": "Johan Bjorck",
4   "s2_author_id": "46278353",
5   "total_publications": 8,
6   "publications_summary": [
7     {
8       "paper_id": "P000001",
9       "year": 2021,
10      "title": "Accelerating Ecological Sciences from Above..."
11    },
12    {
13      "paper_id": "P000832",
14      "year": 2021,
15      "title": "Understanding Decoupled and Early Weight Decay."
16    }
17  ]
18  *****
19  * other papers published *
20  *****
21 ]
22 }

```

Besides the fields name and IDs, more interesting are fields `publication_summary` and `total_publications`: the first one contains a list of papers published by the respective author and the second one is the counter of previous mentioned papers, which is computed and updated at every operation that involves specific author. Finally focus on the format of papers file:

```

1 {
2   "_id": "P000001",
3   "title": "Accelerating Ecological Sciences from Above: Spatial Contrastive Learning
4     for Remote Sensing.",
5   "year": 2021,
6   "dblp_key": "conf/aaai/BjorckRSBDFG21",
7   "doi": "10.1609/aaai.v35i17.17728",
8   "abstract": "The rise of neural networks has opened the door ...",
9   "fields_of_study": [
10     "Computer Science"
11   ],
12   "authors": [

```

³Hashing is generated using `bcrypt` python module method

⁴Partial document embedding is adopted in this case

```

12     {
13         "id": "A00001",
14         "name": "Johan Bjorck"
15     },
16     {
17         "id": "A00002",
18         "name": "B. Rappazzo"
19     }
20 ],
21 "venue": [
22     "AAAI"
23 ]
24 }

```

Once again pay attention particularly on some fields: *authors* and *venue*. First one containing main valuable informations for authors who have written the specific paper (id and name)⁵ and second one contains names of venues where the paper is published.

4.2.3 Neo4j

As it was done for MongoDB, some files are also prepared for graphDB Neo4j. In this case what we need are csv files and to analyze things properly, it is divided the argumentation by nodes and relationships contained in graphDB instance. First of all, let's list 3 kind of nodes that we can face:

1. Paper
2. Author
3. Topic

Follows some example entries in target csv files:

```

# node_authors.csv
authorId:ID(Author),name
A00001,Johan Bjorck

# node_papers.csv
paperId:ID(Paper),title
P000003,What and Where the Themes Dominate in Image.

# node_topics.csv
topicId:ID(Topic),name
T00001,Computer Science

```

As can be seen Nodes have assigned a id and a String, that has different meaning based on node type: name of Author, title of paper or name of Topic. The next step is to present example entries of csv files that constitute informations about relationship:

1. AUTHORED
2. CITES
3. HAS.TOPIC

```

# rels_authored.csv
:START_ID(Author),:END_ID(Paper)
A00001,P000001

# rels_cites.csv
:START_ID(Paper),:END_ID(Paper)
P000001,P016804

# rels_topics.csv
:START_ID(Paper),:END_ID(Topic)
P000001,T00001

```

⁵Partial document embedding

We could say that first relationship involves an author and a paper, in fact consists links an author that has authored a paper; the second relationship connects two papers, where the first paper cites the second one; last relationship connects a paper with its topic.

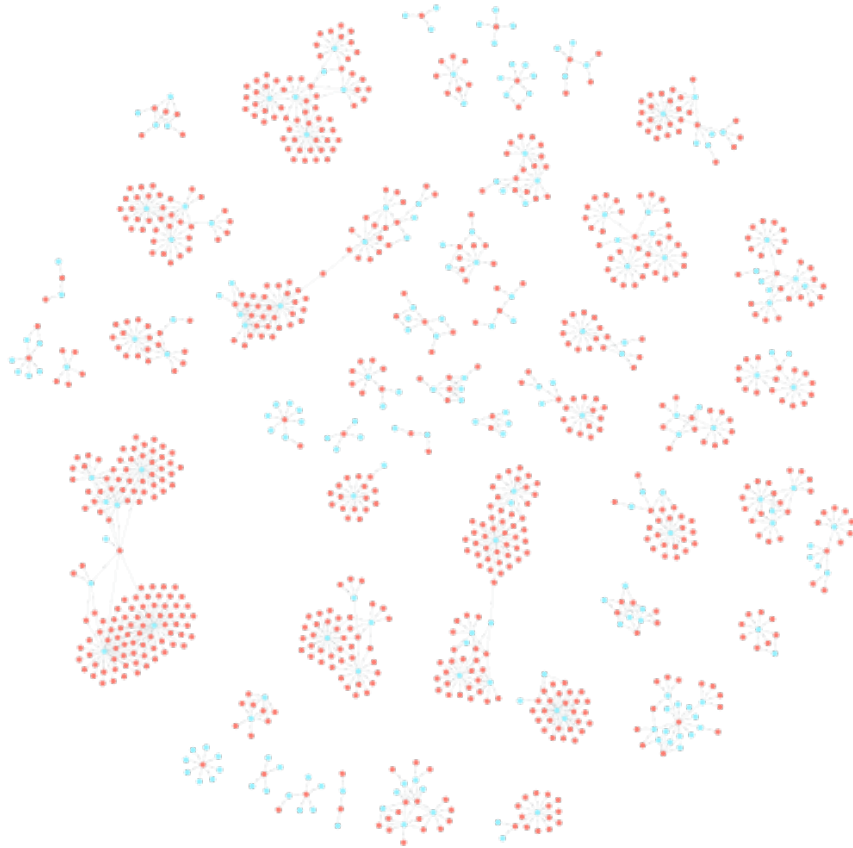


Figure 4.2: AUTHORED subnetwork

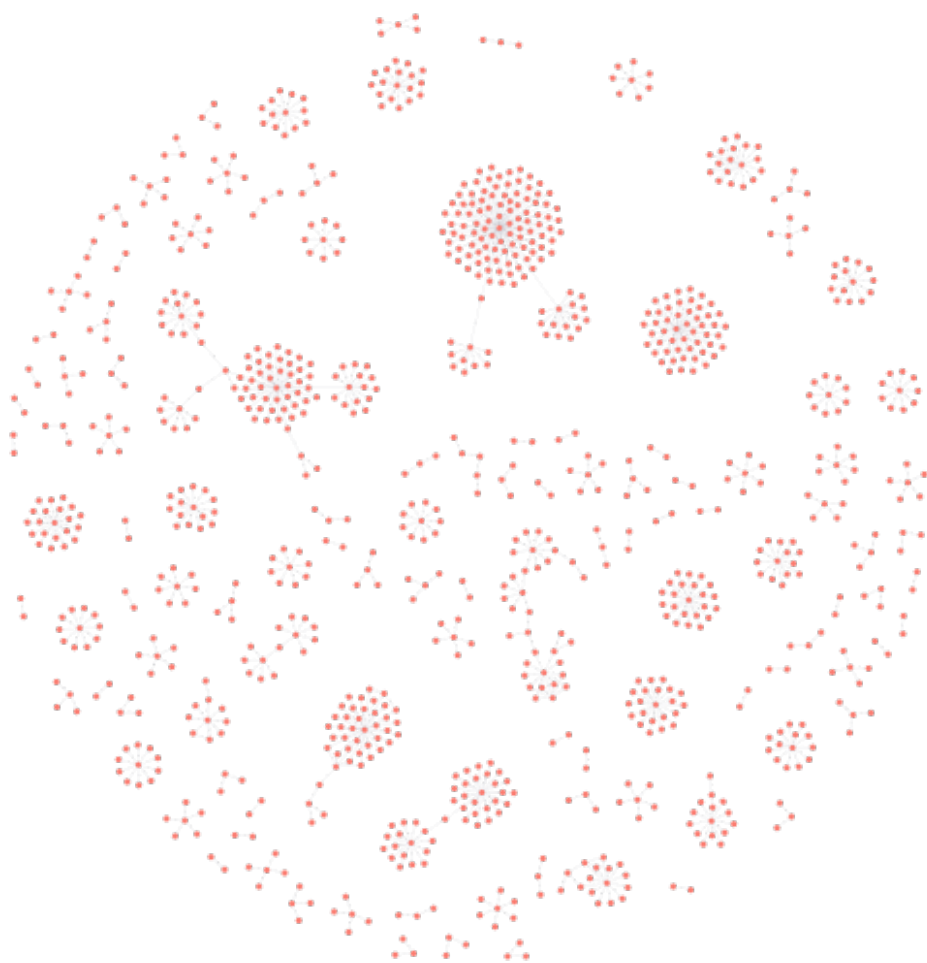


Figure 4.3: CITES subnetwork

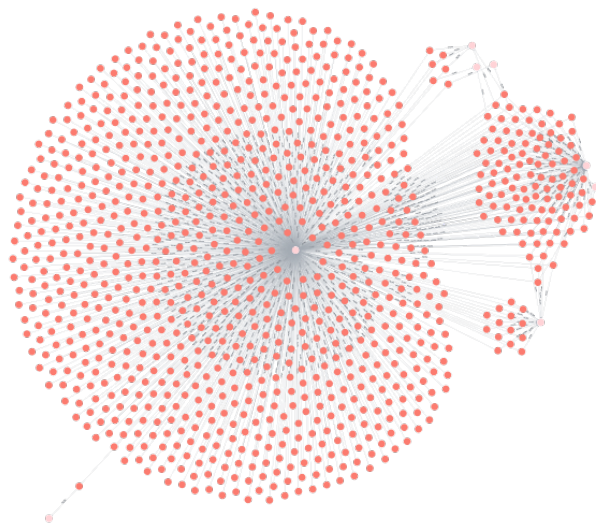


Figure 4.4: HAS_TOPIC subnetwork

Chapter 5

Implementation

This chapter presents the Java classes implemented in the project, organized by package.

5.1 config

OpenApiConfig

Configuration class for OpenAPI (Swagger) documentation. This class uses standard OpenAPI annotations to define the API metadata (title, version) and the security schemes required to access protected endpoints. By defining the SecurityScheme and adding it to the OpenAPIDefinition, the Swagger UI will display an "Authorize" button, allowing users to paste their JWT token and test secured endpoints directly from the browser.

SecurityConfig

Main configuration class for Spring Security. This class configures the application to use stateless authentication via JWT, disables standard web protections that are not needed for REST APIs (like CSRF), and defines the global access rules for HTTP endpoints. It leverages EnableMethodSecurity to allow fine-grained access control directly on service or controller methods using annotations like @PreAuthorize.

5.2 security

JwtTokenFilter

Custom filter that executes once per request to validate JWT tokens. This filter intercepts incoming HTTP requests, extracts the JWT token from the header, validates it using JwtTokenProvider, and sets the authentication in the SecurityContextHolder if the token is valid.

JwtTokenProvider

Component responsible for JWT (JSON Web Token) lifecycle management. Handles token generation, validation, and parsing. Includes a security check against the database to ensure tokens belong to currently active (non-deleted) users.

5.3 controller

AdminController

REST Controller for managing Administrator resources. Provides endpoints for creating, retrieving, updating, and deleting administrator accounts. Includes search functionality and token identity extraction.

AuthController

Controller responsible for handling authentication requests. This controller exposes public endpoints for logging in Administrators and Registered Users. Upon successful authentication, it returns a JWT token that must be included in subsequent requests to protected resources.

AuthorAnalysisController

REST controller for author analytical queries. Provides endpoints to analyze author performance and statistics using MongoDB aggregation pipelines.

AuthorController

REST controller for Author CRUD operations.

PaperAnalysisController

REST controller for paper analytical queries. This controller provides endpoints to retrieve statistical data and trends regarding scientific papers, venues, and collaborations using MongoDB aggregation pipelines.

PaperController

REST controller for Paper CRUD operations.

RegisteredUserAnalysisController

REST controller for managing Registered Users. Provides endpoints for registration, profile updates, retrieval, deletion, search, and bookmark management.

RegisteredUserController

REST controller for managing Registered Users. Provides endpoints for registration, profile updates, retrieval, deletion, and bookmark management.

GraphController

Controller class that provides request mapping for graphDB instance. Here are offered 4 types of get requests:

1. **PageRank**: execute the pagerank algorithm¹ on the graph, considering only cites relationship and papers node only.
2. **Leiden community**: execute the Leiden algorithm in order to obtain several communities where authors are divided in, depending on co-author collaboration.
3. **Betweenness**: execute the betweenness algorithm on papers, in order to retrieve nodes that are identified as bridge nodes with a score (given by result)
4. **Collaboration path**: computed the Shortest Path algorithm in order to obtain the shortest collaboration path that links authors between them through papers and other authors.

5.4 model.mongo

Admin

MongoDB document representing an Administrator in the system. This class extends the base User class to include administrative privileges. Unlike standard users, admins possess specific permissions to manage platform content and user accounts. Data is stored in the separate "admins" collection for security segregation.

¹Offered by Graph Data Science

Author

MongoDB document representing an academic Author. This class maps to the authors collection in the database. It stores the author's personal details, external identifiers, and a summary of their publications to optimize read operations (denormalization).

BookmarkedPaper

Represents a snapshot of a scientific paper bookmarked by a user. This class is designed to be used as an embedded document within RegisteredUser. It stores a lightweight summary of the paper (ID, title, and save date) to facilitate efficient retrieval and display of a user's bookmarks without querying the main papers collection immediately.

CollaborationEvolution

Class representing the evolution of scientific collaboration. Specific metrics regarding the average number of authors per paper for a given year.

Paper

MongoDB document representing an academic paper. This is the core entity of the application, stored in the "papers" collection. It contains bibliographic details, metadata about authors and venues, and classification fields (fields of study).

PaperAuthor

Represents an author embedded within a Paper document. This is a simplified representation of an author used specifically for the context of a single paper.

PaperLeaderboard

An entry in the paper leaderboard. This class is used to map the results of aggregation queries that calculate the popularity of papers based on bookmarks.

Permission

Enumeration defining the specific granular permissions available to Administrators. Permissions define specific authorized actions. These are stored within the Admin document to allow for fine-grained access control over administrative operations.

ProlificAuthor

Class representing the result of a "Prolific Author" analysis. This class is not a direct mapping of a MongoDB collection but a projection result returned by aggregation queries (e.g., identifying authors with high publication counts).

PublicationSummary

Lightweight class representing a summary of a publication. Used when a full paper object is not required, for example in lists or brief citations.

RegisteredUser

MongoDB document representing a standard registered user. This class extends the base User class and adds fields specific to regular members, such as their full name and a personal collection of bookmarked papers. Data is stored in the "registeredUsers" collection.

TrendAnalysis

Class representing the result of a "Hot Topics" trend analysis. Aggregates data to show the volume of papers published in a specific field of study during a specific year.

User

Abstract base class representing a generic user in the Nexus Scholar system. This class captures the common attributes shared by all user types (e.g., Administrators and Registered Users), such as authentication credentials and system metadata. It is not intended to be instantiated directly but rather extended by concrete document classes.

VenueAnalysis

Class representing the result of a Venue Impact analysis. Shows the publication volume of a specific venue (Conference or Journal) for a given year.

5.5 model.neo4j

AuthorNode

Represents an Author entity within the Neo4j Graph Database. Annotated with `@Node`, this class maps to the `Author` label in the graph.

- **Properties:** It stores the `authorId` (as the primary key) and the author's `name`.
- **Relationships:** It defines an outgoing `AUTHORED` relationship to a list of `PaperNode` entities, representing the papers written by this author.

PaperNode

Represents a Scientific Paper entity within the graph. This node acts as the central hub for connections in the network.

- **Properties:** Contains the `paperID` (Primary Key) and `title`.
- **Relationships:**
 - `CITES` (Outgoing): Links to other `PaperNode` entities, representing the citation graph.
 - `HAS.TOPIC` (Outgoing): Links to `TopicNode` entities, categorizing the paper.
 - `AUTHORED` (Incoming): A list of `AuthorNode` entities that authored this paper.

TopicNode

Represents a Field of Study (e.g., "Machine Learning", "Computer Vision"). It contains a `topicId` and a `name`. This node allows the system to perform clustering and trend analysis by aggregating papers connected to specific topics.

5.6 dao

AuthorAnalysisDAO

Data access interface for performing complex analytical queries on Author data. Unlike the standard `AuthorDAO`, this interface handles operations that require MongoDB Aggregation Pipelines (grouping, unwinding, etc.).

PaperAnalysisDAO

Interface defining complex analytical operations on Paper data. These methods typically involve heavy aggregations and grouping operations that go beyond simple CRUD queries.

RegisteredUserAnalysisDAO

Interface defining analytical operations on Registered Users. This DAO focuses on complex queries and aggregations, such as generating leaderboards or statistical analysis of user activities (e.g., bookmarks).

5.7 dao.mongo

AdminDAO

Data Access Object (DAO) for managing Admin entities. Extends MongoRepository to provide standard CRUD operations and custom query methods for the "admins" collection in MongoDB.

AuthorDAO

Spring Data MongoDB repository for Author documents. Provides standard CRUD operations and custom query methods for filtering authors by name, external IDs, and publication metrics.

PaperDAO

Spring Data MongoDB repository for managing Paper documents. Provides standard CRUD operations and custom finder methods with pagination support.

RegisteredUserDAO

Repository interface for managing RegisteredUser entities in MongoDB. Extends MongoRepository to provide standard CRUD operations and custom finder methods optimized for the specific indices defined in the database.

RegisteredUserAnalysisDAOImpl

Implementation of RegisteredUserAnalysisDAO using MongoDB Aggregation Framework.

This class utilizes MongoTemplate to construct and execute complex aggregation pipelines, allowing for efficient data processing directly on the database server.

AuthorAnalysisDAOImpl

MongoDB implementation of the AuthorAnalysisDAO interface.

Uses MongoTemplate to construct and execute aggregation pipelines for analytical tasks regarding authors.

PaperAnalysisDAOImpl

Implementation of PaperAnalysisDAO using MongoDB Aggregation Framework.

This class uses MongoTemplate to construct and execute multi-stage aggregation pipelines.

5.8 dao.neo4j

GraphDAO

This class is provided for executing the offered queries on the graphDB instance, to make it possible an instance of Driver² to create sessions and perform readings and writings (without results) and giving result using utils classes (described later).

5.9 Utils classes

In order to support JSON conversion, all the classes explained in this section support a method *toJson()* that gives a json valid String as return value. In addition getter and setter for attributes and constructor with a Record³ parameter.

²neo4j pre-implemented class

³neo4j class

PageRankEntry

This class models one entry that comes from result set of pagerank algorithm, in particular one entry contains the title of a paper and its assigned score. In this way user could evaluate each paper with its score and in more detail, the greater score is and most relevant a paper is.

LeidenCommunity

This class models the concept of Leiden community, that in our application is composed by an id and a list of authors, that belong to the specific community.

BetweennessEntry

This class models the result entry for betweenness query, which result is composed of paperId, title of paper and betweenness coefficient. As it could be deduced, algorithms works with papers, that in our application are fundamental nodes for linking nodes in the graph, so it would be properly estimate the betweenness and make conclusion on the effect of a removal of a paper.

ShortestPathAuthors

This class models the entry result for ShortestPath query, where we are searching the collaboration path between two authors. Relevant informations are the degree of separation and the path⁴ that we must across to arrive at chosen author node.

NSConstants

This class is developed for unique job to contain constants that are used in Nexus Scholar graph side application: it contains NEO4J_DB, NEO4J_PORT, NEO4J_HOST.

5.10 dao.exception

DAOException

Custom runtime exception for data access layer failures. Wraps underlying database exceptions to provide a consistent exception hierarchy for the service layer to handle.

5.11 dto.auth

AuthResponseDTO

Data Transfer Object (DTO) for the authentication response. This class returns the JWT access token to the client upon successful login.

LoginRequestDTO

Data Transfer Object (DTO) for handling user login requests. This class captures the credentials required for authentication.

5.12 dto.mongo

AdminCreateDTO

DTO containing the data required to create a new Administrator. This class is used exclusively as an input parameter for the creation endpoint. It includes the raw password which must be encrypted by the service layer.

⁴Obtained using Path neo4j class

AdminDTO

DTO representing an Administrator sent in API responses. Extends UserDTO to include admin-specific fields like permissions. Does not contain sensitive security information like the password hash.

AdminUpdateDTO

DTO used for updating an existing Administrator's profile. Allows changing sensitive information such as the password, permissions, and username.

AuthorDTO

Data Transfer Object (DTO) for the Author entity. This class is used to transfer author details to the client, including personal information and a summary of their publication history.

BookmarkedPaperDTO

Lightweight DTO representing a paper saved in a user's bookmark list. Contains minimal information required to display the bookmark on the client side without fetching the full paper document.

CollaborationEvolutionDTO

Data Transfer Object representing the evolution of scientific collaboration over time. This is typically used to graph the average team size (number of authors) per paper per year.

PaperAuthorDTO

Data Transfer Object representing an author associated with a specific paper. This contains a subset of author information typically embedded within a PaperDTO.

PaperDTO

Data Transfer Object (DTO) representing a scientific paper stored in MongoDB. This class encapsulates all relevant metadata regarding a research paper, including its identifiers, bibliographic info, authors, and classification.

PaperLeaderboardDTO

DTO representing an entry in the paper popularity leaderboard. Used to transfer aggregation results, showing which papers have received the most bookmarks within a specific time frame.

PermissionDTO

Enumeration DTO representing the permissions available to an Administrator. Mirrors the internal domain enum to expose permission types over the API without directly leaking the internal implementation.

ProlificAuthorDTO

Data Transfer Object representing the result of a "Prolific Author" analysis. This lightweight class is used to return a list of high-output researchers without including their full publication history.

PublicationSummaryDTO

Data Transfer Object (DTO) representing a summary of a publication. This class is designed to provide a lightweight view of a publication, typically used in lists, search results, or API responses where the full details of the paper are not required.

RegisteredUserCreateDTO

DTO containing the data required to register a new user. This class is used exclusively as an input parameter for the registration endpoint. It includes sensitive information like the raw password, which will be encrypted by the service layer before storage.

RegisteredUserDTO

DTO representing a fully registered user sent in API responses. Extends UserDTO to include specific fields for registered members, such as their full name and their collection of bookmarked papers.

RegisteredUserUpdateDTO

Data Transfer Object (DTO) for updating an existing user's profile. This class encapsulates the fields that a user is allowed to modify. Fields that are null in the request indicate that no change is requested for that specific property.

TrendAnalysisDTO

Data Transfer Object for analyzing publication trends within specific fields of study. This class is used to transport aggregated data indicating the volume of publications for a specific topic over a specific year.

UserDTO

Abstract Data Transfer Object representing the common attributes of any user in the system. Serves as a base class for specific user types (e.g., RegisteredUser, Admin) to ensure consistent field naming for ID, username, and email in API responses.

VenueAnalysisDTO

Data Transfer Object for analyzing publication volume across different venues. This class transports data regarding how many papers were published in a specific venue (conference or journal) during a specific year.

5.13 dto.neo4j

BetweennessDTO

This class is reserved to transport result entries of betweenness query, reporting paper title and score.

LeidenDTO

This class is reserved to transport result entries of Leiden query, that constitute various communities and only authors' name are passed, enough to represent authors in the community.

PageRankDTO

This class is reserved to transport result entries of pagerank query, reporting paper title and score.

ShortestPathDTO

This class is used to transport content of shortestPath query result between two authors. One necessary passage, in order to do serialization in json format, is to divide in two auxiliary classes the content of Path parameter of BetweennessEntry instance. More specifically we retrieve nodes and relationship from the Path class using pre-implemented methods and save in PathNodeDTO and PathRelationshipDTO respectively.

PathNodeDTO

This class is used to transfer Node data that is in a path, in our context represent a paper or an author that are in shortest path between two authors. Parameters of this class are:

- `elementId` : node Id.
- `type` : values "Paper" or "Author" could be assigned.
- `displayName`: value of the String field, retrieved from title if node selected is Paper instance or name if is Author.

PathRelationshipDTO

This class is used to transfer Node data that is in a path, in our context represent a paper or an author that are in shortest path between two authors.

5.14 service

AdminService

Service interface for managing Administrator entities. Defines the business logic for the lifecycle of administrative accounts, including creation, updates, retrieval, search, and deletion.

AuthorAnalysisService

Service interface for author-related analytical operations. This interface defines the business logic for extracting insights about authors, such as identifying prolific researchers based on publication metrics.

AuthService

Service interface for managing Author entities. Handles CRUD operations and maintains data consistency with the Paper collection.

AuthService

Service interface for handling authentication operations. Defines the contract for logging in different types of users (Admins and Registered Users) and retrieving their authentication tokens.

GraphService

All methods provided for graph controller, converting result that comes from graphDAO instance into DTO instances and construct the body of the response to the request made by the user. Considering that content of response could easily assume very huge volumes, it is supported the Page feature, that simply limit the response body dimensions using *Pageable* instance, from which are retrieved skip and limit values, then used in queries in order to limit result set. This class also contains methods to convert model class instances in respective dto class instances (i.e. *BetweennessEntry* into *BetweennessDTO*). Logic supported regards page ranking of papers, computations for collaboration path for 2 authors, betweenness of a paper node and Leiden hidden communities.

HealthService

This class has the role to respond when system is working properly, in fact there is only one method that return string that semantically means system is working.

PaperAnalysisService

Service interface for paper-related analytical operations. Decouples the controller from the complex aggregation logic implemented in the DAO layer. Transforms raw statistical data into DTOs suitable for charts and reports.

PaperService

Service interface for managing Paper entities. Handles CRUD operations and maintains bidirectional consistency with Authors.

RegisteredUserAnalysisService

Service interface for analytical operations related to Registered Users.

This service handles high-level logic for generating statistics and leaderboards, decoupling the controller from the data access layer.

RegisteredUserService

Service interface for managing Registered User entities. Defines the business logic for user registration, profile updates (including sensitive data like passwords), information retrieval, and search capabilities.

5.15 service.exception

BusinessException

Custom runtime exception for business logic violations. Used when operations fail due to domain rules rather than technical errors. Examples: duplicate entries, invalid state transitions, constraint violations.

GlobalExceptionHandler

Centralized exception handler for the REST API. Intercepts exceptions thrown by Controllers or Services and returns standardized JSON error responses with appropriate HTTP status codes.

5.16 service.impl

AdminServiceImpl

Implementation of the AdminService interface. Handles business logic for Admin management, including unique constraint checks for username and email updates.

AuthorAnalysisServiceImpl

Implementation of the AuthorAnalysisService. This class acts as a bridge between the REST controller and the DAO layer. It retrieves raw analysis data from the database and maps it to DTOs for the client.

PaperAnalysisServiceImpl

Implementation of PaperAnalysisService. Bridges the gap between the DAO's raw aggregation results (Entity/Model objects) and the API's required format (DTOs).

RegisteredUserAnalysisServiceImpl

Implementation of the RegisteredUserAnalysisService. This class coordinates the retrieval of raw aggregation results from the DAO and transforms them into Data Transfer Objects (DTOs) suitable for the API response.

AuthServiceImpl

Implementation of the AuthService interface. This service handles business logic for managing Author entities in MongoDB. It is responsible for maintaining data consistency between the Author collection and the Paper collection, particularly when updating publication lists or deleting authors.

PaperServiceImpl

Implementation of the PaperService interface. This service manages the lifecycle of Paper entities. It is responsible for: CRUD operations on Papers in MongoDB. Synchronizing paper data with the Neo4j Graph Database via GraphDAO. Maintaining consistency with the Author collection (e.g., updating author publication counts).

AuthServiceImpl

Implementation of the AuthService interface. This class handles the business logic for authentication, including:

- Retrieving user or admin entities from the database.
- Verifying raw passwords against stored encrypted passwords.
- Generating JWT tokens upon successful authentication using JwtTokenProvider.

RegisteredUserServiceImpl

Implementation of RegisteredUserService. Handles core business logic including password hashing, uniqueness validation, and interaction with the data access layer.

Chapter 6

Relevant Queries

6.1 MongoDB

6.1.1 Venue Impact Analysis

To understand the landscape of scientific publishing, the system includes a “Venue Impact Analysis” query. This feature ranks publication venues (conferences, journals, workshops) based on their output volume within a specific solar year. This allows researchers to identify the most active and dominant venues over time.

1. Aggregation Logic

The process involves normalizing venue data and aggregating it temporally. The pipeline steps are defined as follows:

1. **Normalization (\$unwind):** Since the schema allows for flexibility (a paper might be associated with multiple venue tags or nested structures), the `venue` array is first unwound. This ensures that every association between a paper and a venue is treated as a distinct record for counting.
2. **Compound Grouping:** The core metric is calculated by grouping documents based on a composite key: the `venue` name and the `year` of publication. This separates the performance of a venue in 2023 from its performance in 2024.
3. **Volume Aggregation:** A count aggregation (`$sum`) is applied to determine the total number of papers published in each specific venue-year pair.
4. **Hierarchical Sorting:** The results are sorted first chronologically (by Year ascending) and then by volume (by Count descending). This structure produces a list where, for every year, the most popular venues appear at the top.

2. MongoDB Query (Shell)

The following shell command illustrates how the venue data is flattened and grouped to produce the ranked list.

Listing 6.1: Aggregation pipeline for Venue Analysis

```
1 db.papers.aggregate([
2   {
3     $unwind: "$venue"
4   },
5   {
6     $group:
7     {
8       _id: { venue: "$venue", year: "$year"},
9       paper_created: {$sum : 1}
10    }
11  },
12  {
13    $sort: {"_id.year":1, paper_created:-1}
```

```

14     },
15     {
16         $project: {
17             "_id" : 0,
18             "venue" : "$_id.venue",
19             "year" : "$_id.year",
20             "paper_created": "$paper_created"
21         }
22     }
23 })

```

3. Java Implementation (Spring Data)

The application layer executes this logic via the `getVenueAnalysis` method. The result is mapped to a list of `VenueAnalysis` objects, which are suitable for displaying in tabular or chart formats on the frontend.

Listing 6.2: Java implementation for ranking venues

```

1 public List<VenueAnalysis> getVenueAnalysis() throws DAOException {
2     try {
3         Aggregation aggregation =
4             Aggregation.newAggregation(
5                 // 1. Explode venue array
6                 Aggregation.unwind("venue"),
7
8                 // 2. Group by venue and year
9                 Aggregation.group("venue", "year").count().as("paper_created"),
10
11                 // 3. Project to flatten
12                 Aggregation.project("paper_created")
13                     .and("_id.venue")
14                     .as("venue")
15                     .and("_id.year")
16                     .as("year")
17                     .andExclude("_id"),
18
19                 // 4. Sort
20                 Aggregation.sort(
21                     Sort.by(Sort.Direction.ASC, "year")
22                     .and(Sort.by(Sort.Direction.DESC, "paper_created"))));
23
24         return mongoTemplate.aggregate(aggregation, "papers", VenueAnalysis.class).
25             getMappedResults();
26     } catch (Exception e) {
27         throw new DAOException("Error executing venue analysis aggregation", e);
28     }
29 }

```

6.1.2 Collaboration Evolution Analysis

To analyze how scientific collaboration patterns have changed over time, the system implements a “Collaboration Evolution” metric. This analysis calculates the average number of authors per paper for each year. An upward trend in this metric typically indicates a shift towards larger research teams and more complex, cross-disciplinary studies.

1. Aggregation Logic

This computation requires deriving a numerical metric from an array before grouping. The pipeline execution flow is as follows:

- Array Size Calculation (\$project):** The first step is to transform the document structure. Instead of passing the full array of author objects, the system calculates the size of the `authors` array using the `$size` operator. This creates a temporary field, `num_authors`, for every paper.
- Temporal Grouping:** The documents are grouped by `year`. This creates a bucket for every year present in the dataset.

3. **Average Computation:** Within each yearly bucket, the system calculates the arithmetic mean (\$avg) of the num_authors fields. This produces the average team size for that specific year.
4. **Sorting and Formatting:** The resulting list is sorted chronologically (ascending year) to allow for linear plotting on a graph. The final projection aligns the field names with the application's data transfer object.

2. MongoDB Query (Shell)

The following shell script demonstrates the use of the \$size operator within the projection stage to prepare the data for the subsequent grouping.

Listing 6.3: Aggregation pipeline for Collaboration Evolution

```

1 Collaboration Evolution: Calculating the average number of authors per paper for each
  year
2 db.papers.aggregate([
3   {
4     $project:
5     {
6       year: "$year",
7       num_authors: {$size: "$authors"}
8     }
9   },
10  {
11    $group:
12    {
13      _id: {year: "$year"},
14      avg_authors: {$avg: "$num_authors"}
15    }
16  },
17  {
18    $sort: {"_id.year": 1}
19  },
20  {
21    $project: {
22      _id: 0,
23      year: "$_id.year",
24      avg_authors: "$avg_authors"
25    }
26  }
27 ])
```

3. Java Implementation (Spring Data)

The Java implementation utilizes Spring Data's arithmetic operators to compute the array size. The getCollaborationEvolution method returns a chronological list of CollaborationEvolution objects.

Listing 6.4: Java implementation for calculating Average Authors per Year

```

1 public List<CollaborationEvolution> getCollaborationEvolution() throws DAOException {
2     try {
3         Aggregation aggregation =
4             Aggregation.newAggregation(
5                 // 1. Project year and compute author count per paper
6                 Aggregation.project("year").and("authors").size().as("num_authors"),
7
8                 // 2. Group by year and compute average authors
9                 Aggregation.group("year").avg("num_authors").as("avg_authors"),
10
11                 // 3. Project to adjust field names
12                 Aggregation.project("avg_authors").and("_id").as("year").andExclude("_id")
13                 ,
14
15                 // 4. Sort by year ascending
16                 Aggregation.sort(Sort.Direction.ASC, "year"));
17
18         return mongoTemplate
19             .aggregate(aggregation, "papers", CollaborationEvolution.class)
20             .getMappedResults();
21     }
22 }
```

```

20
21     } catch (Exception e) {
22         throw new DAOException("Error executing collaboration evolution aggregation", e);
23     }
24 }

```

6.1.3 Trend Analysis and Hot Topics

To identify emerging research trends, the system implements a “Trend Analysis” feature. This process calculates the publication volume per field of study over time, allowing the application to visualize which topics are gaining traction (“Hot Topics”) and how research focus has shifted across different years.

1. Aggregation Logic

Unlike simple counting, this operation must handle the many-to-many relationship between papers and their fields of study. A single paper often belongs to multiple fields (e.g., “Machine Learning” and “Healthcare”). The aggregation logic proceeds as follows:

1. **Array Decomposition (\$unwind):** The `fields_of_study` attribute is an array. To count a paper towards *all* its relevant topics, the array is unwound. This replicates the paper document for each field of study it contains.
2. **Composite Grouping:** The data is grouped using a composite key consisting of both the `field_of_study` and the `year`. This granularity is essential to track the evolution of a specific topic on a yearly basis.
3. **Volume Calculation:** Within each group, a summation allows the system to calculate the total number of papers created for that specific topic-year combination.
4. **Projection and Sorting:** Finally, the composite ID is flattened into a readable format, and the results are sorted chronologically by year and by volume, ensuring the data is ready for visualization charts.

2. MongoDB Query (Shell)

The following pipeline demonstrates how the multi-valued fields are processed to generate the trend metrics.

Listing 6.5: Aggregation pipeline for Trend Analysis

```

1 db.papers.aggregate([
2     {
3         $unwind: "$fields_of_study"
4     },
5     {
6         $group:
7         {
8             _id: {field_of_study:"$fields_of_study", year:"$year"},
9             paper_created: {$sum:1}
10        }
11    },
12    {
13        $sort: {"_id.year":1, "paper_created" :-1}
14    },
15    {
16        $project : {
17            _id:0,
18            field_of_study: "$_id.field_of_study",
19            year: "$_id.year",
20            paper_created: "$paper_created"
21        }
22    }
23 ])

```

3. Java Implementation (Spring Data)

The Java implementation encapsulates this logic within a method that returns a list of `TrendAnalysis` DTOs. It uses Spring Data’s fluent API to construct the pipeline and includes error handling to manage potential data access issues.

Listing 6.6: Java implementation for extracting Research Trends

```
1 public List<TrendAnalysis> getTrendAnalysis() throws DAOException {
2     try {
3         Aggregation aggregation =
4             Aggregation.newAggregation(
5                 // 1. Explode fields_of_study array
6                 Aggregation.unwind("fields_of_study"),
7
8                 // 2. Group by field of study and year
9                 Aggregation.group("fields_of_study", "year").count().as("paper_created"),
10
11                 // 3. Project to flatten the composite _id
12                 Aggregation.project("paper_created")
13                     .and("_id.fields_of_study")
14                     .as("field_of_study")
15                     .and("_id.year")
16                     .as("year")
17                     .andExclude("_id"),
18
19                 // 4. Sort results
20                 Aggregation.sort(
21                     Sort.by(Sort.Direction.ASC, "year")
22                     .and(Sort.by(Sort.Direction.DESC, "paper_created"))));
23
24         return mongoTemplate.aggregate(aggregation, "papers", TrendAnalysis.class).
25             getMappedResults();
26     } catch (Exception e) {
27         throw new DAOException("Error executing trend analysis aggregation", e);
28     }
29 }
```

6.1.4 Prolific Author Identification

The system includes a feature to identify “Prolific Authors,” defined as researchers who have published a significant volume of work within a single solar year. This query is parameterizable, allowing the system to set a specific threshold (e.g., more than X papers) to filter for high-output individuals.

1. Aggregation Logic

Identifying these authors requires a multi-stage grouping process to calculate annual productivity before filtering. The logic is as follows:

1. **Decomposition (\$unwind):** The `publications_summary` is an array containing a summary of an author’s activity. This array is unwound to treat each publication record as a discrete element for processing.
2. **Annual Productivity Calculation:** The first grouping stage uses a compound key consisting of the `Author ID`, `Name`, and the `Year` of publication. The system counts the occurrences in each group, effectively calculating how many papers a specific author published in a specific year.
3. **Threshold Filtering (\$match):** A filter is applied to the calculated count. Only groups where the number of created papers strictly exceeds the provided threshold (e.g., $N > 5$) are retained.
4. **Deduplication:** An author might meet the criteria in multiple different years (e.g., being prolific in both 2019 and 2021). To ensure the final list of authors is unique, a second grouping stage is applied on the `Author ID` to consolidate duplicates.
5. **Projection:** The final stage formats the output to match the Data Transfer Object (DTO), returning only the ID and Name of the identified authors.

2. MongoDB Query (Shell)

The following pipeline demonstrates the double-grouping strategy required to calculate counts per year and then distinct authors.

Listing 6.7: Aggregation pipeline for Prolific Authors

```
1 db.authors.aggregate([
2   {
3     $unwind: "$publications_summary"
4   },
5   {
6     $group:
7     {
8       _id: {$_id: "$_id", name: "$name", year: "$year"},
9       paper_created: {$sum:1}
10    }
11  },
12  {
13    $match:
14    {
15      paper_created: {$gt: 5}
16    }
17  },
18  {
19    $group:
20    {
21      _id: "$_id._id",
22      name: {$first: "$_id.name"}
23    }
24  },
25  {
26    $project : {
27      _id:0,
28      authorId: "$_id",
29      authorName: "$name"
30    }
31  }
32 ])
```

3. Java Implementation (Spring Data)

The Java implementation accepts a dynamic integer parameter `minPublications`. This allows the application to adjust the strictness of the "prolific" definition at runtime.

Listing 6.8: Java implementation for filtering Prolific Authors

```
1 public List<ProlificAuthor> getProlificAuthors(int minPublications) throws DAOException
2 {
3     try {
4         Aggregation aggregation =
5             Aggregation.newAggregation(
6                 // 1. Explode the publications array to analyze each entry individually
7                 Aggregation.unwind("publications_summary"),
8
9                 // 2. Group by Author (ID + Name) and Year to calculate annual
10                productivity
11                Aggregation.group("_id", "name", "publications_summary.year")
12                .count()
13                .as("paper_created"),
14
15                // 3. Filter results to keep only year/author pairs meeting the threshold
16                Aggregation.match(Criteria.where("paper_created").gt(minPublications)),
17
18                // 4. Re-group to eliminate duplicate author records (if they were
19                prolific in
20                // multiple years)
21                Aggregation.group("_id._id").first("_id.name").as("author_name"),
22
23                // 5. Project the final fields to match the ProlificAuthor DTO
24                Aggregation.project("author_name").and("_id").as("author_id"));
```

```

23     return mongoTemplate
24         .aggregate(aggregation, "authors", ProlificAuthor.class)
25         .getMappedResults();
26
27     } catch (Exception e) {
28         throw new DAOException("Error executing prolific authors aggregation", e);
29     }
30 }

```

6.1.5 PaperLeaderboard

To determine the leaderboard of the “Most Bookmarked Papers” for a desired month, a data processing pipeline was defined. The process follows a three-tiered approach: logical definition, database query, and application implementation.

1. Aggregation Logic

Calculating the leaderboard requires transforming user-centric data into paper-centric metrics. The sequential logic applied is as follows:

1. **Projection and Unwind:** Initially, the system extracts the array of saved papers from each registered user. Since the relationship is *one-to-many* (one user has multiple bookmarks), the array is denormalized using the `$unwind` stage. This transforms every bookmark into an independent document to facilitate counting.
2. **Temporal Filtering:** A strict filter is applied to isolate only the activities that occurred in the requested year and month (e.g., November 2025). This step drastically reduces the dataset size before the grouping phase.
3. **Grouping and Counting:** The remaining documents are grouped by the unique paper identifier (`paper_id`). For each group, an incremental counter is calculated, representing the total number of users who have saved that specific paper.
4. **Ranking:** Finally, the results are sorted in descending order based on the bookmark count, and the output is limited to the top 5 results.

2. MongoDB Query (Shell)

The logic described above translates into the following native MongoDB *Aggregation Pipeline*. Note the use of `$expr` within the match stage to handle date component extraction (year and month) directly on the server side.

Listing 6.9: MongoDB aggregation pipeline for the leaderboard

```

1 db.registeredUsers.aggregate([
2     {
3         $project : {
4             _id : 0,
5             bookmarked_paper : "$bookmarked_papers"
6         }
7     },
8     {
9         $unwind: "$bookmarked_paper"
10    },
11
12    {
13        $match: {
14            $expr: {
15                $and: [
16                    { $eq: [ { $year: "$bookmarked_paper.saved_at" }, 2025 ] },
17                    { $eq: [ { $month: "$bookmarked_paper.saved_at" }, 11 ] }
18                ]
19            }
20        }
21    },
22    {
23        $group :{

```

```

24         _id:{
25             paper_id : "$bookmarked_paper.paper_id",
26             title: "$bookmarked_paper.title"
27         },
28         bookmarked_received: {$sum:1}
29     }
30 },
31 {
32     $project : {
33         _id : 0,
34         paper_id : "$_id.paper_id",
35         title: "$_id.title",
36         bookmarked_received : "$bookmarked_received"
37     }
38 },
39 {
40     $sort :{"bookmarked_received":-1}
41 },
42 {
43     $limit: 5
44 }
45 ])

```

3. Java Implementation (Spring Data)

At the application level, the query is constructed programmatically using `MongoTemplate` and the `Aggregation` class. This approach ensures strong typing and allows for the dynamic parameterization of the target year and month.

Listing 6.10: Java implementation using Spring Data MongoDB

```

1  public List<PaperLeaderboard> getMostBookmarkedPapers(int year, int month) {
2
3      Aggregation aggregation =
4          newAggregation(
5              // 1. Project: Rename 'bookmarked_papers' array to singular '
6              //      bookmarked_paper'
7              project().and("bookmarked_papers").as("bookmarked_paper").andExclude("_id"),
8
9              // 2. Unwind: Deconstruct the array to process individual bookmarks
10             unwind("bookmarked_paper"),
11
12             // 3. Match: Filter papers saved in the specific Year and Month
13             match(
14                 Criteria.expr(
15                     BooleanOperators.And.and(
16                         ComparisonOperators.valueOf(
17                             DateOperators.Year.yearOf("bookmarked_paper.saved_at"))
18                             .equalToValue(year),
19                         ComparisonOperators.valueOf(
20                             DateOperators.Month.monthOf("bookmarked_paper.saved_at")
21                             )
22                             .equalToValue(month))),
23
24             // 4. Group: Count bookmarks per paper (grouping by ID and Title)
25             group("bookmarked_paper.paper_id", "bookmarked_paper.title")
26                 .count()
27                 .as("bookmarked_received"),
28
29             // 5. Project: Format the output to match PaperLeaderboard class structure
30             project()
31                 .and("_id.paper_id")
32                 .as("paper_id")
33                 .and("_id.title")
34                 .as("title")
35                 .and("bookmarked_received")
36                 .as("bookmarked_received")
37                 .andExclude("_id"),
38
39             // 6. Sort: Order by most bookmarks (Descending)
40             sort(Sort.Direction.DESC, "bookmarked_received"),
41
42         )
43     }
44 }

```

```

40         // 7. Limit: Get top 5 results
41         limit(5));
42
43     return mongoTemplate
44         .aggregate(aggregation, "registeredUsers", PaperLeaderboard.class)
45         .getMappedResults();
46 }

```

6.2 Neo4j

Our goal is to give explanation of each query starting from domain point of view and then obtain the respective solution in graph context. For a complete explanation, Java code segments will be given in addition segments that are responsible of executing query steps.

6.2.1 Page ranking papers

Suppose that a user would like to make a ranking of papers, that maybe wants to read or simply explore that relative field of study. So it is worth thinking about what the factors are that imply an "importance" of a paper and make it "better" than others, especially how to represent this result and show to users. Here we can introduce one fundamental concept: A paper is more relevant if a lot of papers cite it, because if a lot of papers cited it its content is more interesting and valuable (as we supposed). So we can state that if one paper is "better" than another one means that the first one has more citations than second one. So now let's move on questioning how can we determine the importance coefficient of a single paper considering citations that involves it: it is necessary to make evident that we're not interested in all data, just at papers and citations. So move now to the graph domain: we've modelled papers as nodes and there is also a relationship that models citations phenomena (:CITES) and for measure what we're interested in it is necessary to consider incoming edges of a paper node. Here it comes to help the **page rank algorithm**, that measures importance of a node considering the incoming edges of a node¹. Finally in order to retrieve all result entries, java method uses PageRankEntry instances (described in previous chapter). So it follows neo4j queries that are necessary to obtain the result explained above and its java corresponding code.

Neo4j page rank query

As it was said before, we've to consider only a partition of data of the graph and in order to retrieve them we use a graph projection, which filters paper type nodes and :CITES relationship edges:

```

1 CALL gds.graph.project(
2     'paperCitations',
3     'Paper',
4     'CITES'
5 );

```

To spend some words on this code: it project from the main graph node of Type Paper and relationship :CITES and save on in-memory result paperCitations. Then we can execute the effective query:

```

1 CALL gds.pageRank.stream('paperCitations')
2 YIELD nodeId, score
3 RETURN gds.util.asNode(nodeId).title as title, round(score,4) as rank
4 ORDER BY score DESC
5 SKIP $s
6 LIMIT $l;

```

To clarify use of this query, YIELD part permits to filter parameters that are computed for page rank algorithm, in fact we're interested only on nodeId and score values, then are retrieved title of paper and ranking computed, where the last will be used as sorting parameter. As it could be seen SKIP and LIMIT values are assigned using a parameter, this is because output assume considerable dimensions and needs to be limited.

¹More details on Neo4j Graph Data Science official doc

Java code page rank query

In addition to what is said before, java code manage other things:

- if projection already exists it is not created again, in this way it is improved performance of operation.
- Once list of Record is retrieved, will be created a list of PageRankEntry to return to service class (that will be converted in list of PageRankDTO instances)

```
1 public List<PageRankEntry> pageRankAlg(int skip, int limit) {
2     try (Session session = driver.session()) {
3         boolean ex =
4             session.executeRead(
5                 tx -> {
6                     Result res =
7                         tx.run("CALL gds.graph.exists('paperCitations') YIELD exists RETURN
8                             exists");
9                     return res.single().get("exists").asBoolean();
10                });
11
12        if (!ex) {
13            session.executeWriteWithoutResult(
14                tx -> {
15                    tx.run(
16                        """
17                        CALL gds.graph.project(
18                            'paperCitations',
19                            'Paper',
20                            'CITES'
21                        );
22                        """
23                    );
24                });
25
26        List<org.neo4j.driver.Record> lr =
27            session.executeRead(
28                tx -> {
29                    Result res =
30                        tx.run(
31                            """
32                            CALL gds.pageRank.stream('paperCitations')
33                            YIELD nodeId, score
34                            WITH nodeId, score
35                            ORDER BY score DESC
36                            SKIP $s
37                            LIMIT $l
38                            RETURN gds.util.asNode(nodeId).title as title, round(score,
39                                4) as rank;
40                            """
41                            ,
42                            Map.of("s", skip, "l", limit));
43                    return res.list();
44                });
45
46        return lr.stream().map(PageRankEntry::new).collect(Collectors.toList());
47    } catch (Exception e) {
48        log.error("PageRank calculation failed", e);
49        return Collections.emptyList();
50    }
```

For better reading and good-manner coding, streams were used in order to iterate the list of records and created list of pageRankEntry.

6.2.2 Shortest Collaboration Path

Imagine that you wonder how far apart two authors are in context of researchers' world, that means to verify the degree of separation between two selected authors. This computations are useful in order to link two different people that has maybe never co-authored together. Obviously there could be a lot of ways to link two people and this fact could lead us to have several values for degree of separation:

objective of this query is to find the lower one, because gives me more meaningful informations about two targeted authors. In this case, we face a query that doesn't need to work with all data we dispose, in fact only interesting data are composed by authors and their publications (papers), because papers connect author one with others.

Neo4j shortestPath query

Let's focus now on graph-domain: as mentioned before not all data are useful to compute the degree of separation, in fact only authors, papers and relationship :AUTHORED is necessary for our goal. Once we've retrieved all above elements, we have only to call the **shortest path algorithm** and consider first path given (using YIELD clause) and then compute the degree of separation, which is computed calculating the length of path and divided it by 2 because for each step of co-author path we've an author node and one paper node, so that quotient is necessary to not overestimate quantity desired.

```

1 MATCH (a1:Author{name:$a1Name}),
2     (a2:Author{name:$a2Name}),
3     path=shortestPath(
4         (a1) - [:AUTHORED*..10] - (a2)
5     )
6 RETURN path,
7     length(path)/2 as DegreeSeparation
8 ORDER BY DegreeSeparation ASC
9 LIMIT 1

```

Java code shortestPath query

For not degrade performance, number of edges for :AUTHORED relationship check is limited to 10, in fact giving enough data for algorithm. Another observation can be done with the name of two authors, which are binded to the query. Then it is reported the full java code that handles the shortest path logic:

```

1 public ShortestPathAuthors shortestPathAlg(String author1, String author2) {
2     if (author1 == null || author1.isEmpty() || author2 == null || author2.isEmpty())
3         return null;
4
5     try (Session session = driver.session()) {
6         Record r =
7             session.executeRead(
8                 tx -> {
9                     Result res =
10                         tx.run(
11                             """
12                             MATCH (a1:Author{name:$a1Name}),
13                                 (a2:Author{name:$a2Name}),
14                                 path=shortestPath(
15                                     (a1) - [:AUTHORED*..10] - (a2)
16                                 )
17
18                             RETURN path,
19                                     length(path)/2 as DegreeSeparation
20                             ORDER BY DegreeSeparation ASC
21                             LIMIT 1
22                             """,
23                             Map.of("a1Name", author1, "a2Name", author2));
24                         return res.hasNext() ? res.single() : null;
25                     });
26
27         if (r != null) return new ShortestPathAuthors(r);
28
29         return null;
30     } catch (Exception e) {
31         log.error("Shortest path calculation failed", e);
32         return null;
33     }
34 }

```

6.2.3 Hidden Leiden Communities

In the world of researchers, it could be useful divide all authors into different communities to discover different school of thought and collaborate frequently. Obviously knowing this information could help the user to receive a better service from the application and also learn about a topic comparing multiple sources.

Neo4j Leiden query

In this case isn't necessary to work collecting all data in the graph, but only a subset of infos: authors, papers and AUTHORED relationship. In order to do so, it is necessary to work on a projection, which code is the following

```
1 CALL gds.graph.project(  
2     'coAuthors',  
3     ['Author', 'Paper'],  
4     {  
5         AUTHORED: {  
6             orientation: 'UNDIRECTED'  
7         }  
8     }  
9 )
```

The reason why relationship orientation is computed as UNDIRECTED is determined by the fact that Leiden algorithm (offered by GDS) requires to work on graph projection with undirected edges. Then here it is the main query that gives us various communities:

```
1 CALL gds.leiden.stream('coAuthors', { randomSeed: 42 })  
2 YIELD nodeId, communityId  
3 WITH communityId, gds.util.asNode(nodeId) as node  
4 WHERE node:Author  
5 WITH communityId, collect(node.name) as authors  
6 RETURN communityId, authors  
7 ORDER BY communityId ASC  
8 SKIP $skip  
9 LIMIT $limit
```

Let's comment this expression: Leiden algorithm returns us both the nodeId and communityId, but we have to filter nodes, considering only Author kind nodes, because are main entities we want to consider. Obviously to organize better the result set, what we have to collect with name of authors that belong to same community (same communityId) in order to get a record like: communityId,[author1,author2,...authorN], permitting to use utils class that represents query record result. Pagination of query is also supported due to considerable dimensions of result of this query (notice SKIP and LIMIT).

Java code leiden query

```
1 public List<LeidenCommunity> leidenCommunityAlg(int skip, int limit) {  
2     try (Session session = driver.session()) {  
3         ensureCoAuthorsGraph(session);  
4  
5         return session.executeRead(  
6             tx -> {  
7                 Result r =  
8                     tx.run(  
9                         ""  
10                        CALL gds.leiden.stream('coAuthors', { randomSeed: 42 })  
11                        YIELD nodeId, communityId  
12                        WITH communityId, gds.util.asNode(nodeId) as node  
13                        WHERE node:Author  
14                        WITH communityId, collect(node.name) as authors  
15                        RETURN communityId, authors  
16                        ORDER BY communityId ASC  
17                        SKIP $skip  
18                        LIMIT $limit
```

```

19         "",
20         Map.of("skip", skip, "limit", limit));
21
22     return r.list().stream()
23         .map(
24             record ->
25                 new LeidenCommunity(
26                     record.get("communityId").asInt(),
27                     record.get("authors").asList(Value::asString))
28             .collect(Collectors.toList()));
29     });
30 } catch (Exception e) {
31     log.error("Leiden Community detection failed", e);
32     return Collections.emptyList();
33 }
34 }

```

To return result is built a list of LeidenCommunity that contains community Id and authors that belong to the community. In addition to this method, another one check and, if needed, creates the projection used by this algorithm:

```

1 private void ensureCoAuthorsGraph(Session session) {
2     boolean exists =
3         session.executeRead(
4             tx -> {
5                 Result res = tx.run("CALL gds.graph.exists('coAuthors') YIELD exists
6                     RETURN exists");
7                 return res.single().get("exists").asBoolean();
8             });
9
10    if (!exists) {
11        session.executeWriteWithoutResult(
12            tx -> {
13                tx.run(
14                    """
15                    CALL gds.graph.project(
16                        'coAuthors',
17                        ['Author', 'Paper'],
18                        {
19                            AUTHORED: {
20                                orientation: 'UNDIRECTED'
21                            }
22                        }
23                    """
24                );
25            });
26    }

```

6.2.4 Betweenness of papers

As it could be noticed, world of researchers is linked by papers, these elements permit to link author, link other papers with citations and are part of macro topics. So, given this importance, it is useful to estimate the contribute that each paper gives to linking people and papers, and especially considering that several papers or authors are uniquely associable thanks to one specific paper, same for topics too.

Neo4j betweenness query

What we're searching for are, in fact, papers that can be classified as bridge nodes, where one of this is the only way to reach a particular part of the graph (fundamental for queries) but also it can't be removed easily, because would create disjoint graphs and other analytics could be intensely affected. So it is properly to compute a coefficient that measures influence of a node in the flow of informations: betweenness algorithm² helps us perfectly. To make this possible, it is necessary to provide in-memory projection of paperCitations (explained at listing 6.2.1) then we can execute the following query:

²More details on <https://neo4j.com/docs/graph-data-science/current/algorithms/betweenness-centrality/Neo4j> Graph Data Science official doc

```

1 CALL gds.betweenness.stream('paperCitations', {
2                                     samplingSize: 1000,
3                                     samplingSeed: 42
4                                     })
5 YIELD nodeId, score
6 RETURN gds.util.asNode(nodeId).title AS title,
7        round(score, 4) AS betweenness
8 ORDER BY score DESC
9 SKIP $skip
10 LIMIT $limit

```

Notice that algorithm takes as input two additional parameter `samplingSeed` and `samplingSize`, that represent number of nodes to compute betweenness centrality and seed for random number generator respectively.

Java code betweenness query

Here it follows code that manage operations to execute betweenness query: an additional control is done to verify that projection contains some nodes and some relationship by returning `nodeCount` and `relationshipCount` from `list` method (from `gds`), otherwise betweenness algorithm will produce non reliable results.

```

1 public List<BetweennessEntry> betweennessAlg(int skip, int limit) {
2     try (Session session = driver.session()) {
3
4         // 1. Ensure the Graph Projection exists
5         boolean exists =
6             session.executeRead(
7                 tx -> {
8                     Result res =
9                         tx.run("CALL gds.graph.exists('paperCitations') YIELD exists RETURN
10                                exists");
11                     return res.single().get("exists").asBoolean();
12                 });
13
14         if (!exists) {
15             session.executeWriteWithoutResult(
16                 tx -> {
17                     tx.run(
18                         """
19                         CALL gds.graph.project(
20                             'paperCitations',
21                             'Paper',
22                             'CITES'
23                         )
24                         """
25                     );
26                 });
27
28         // 2. Check for empty graph
29         Record stats =
30             session.executeRead(
31                 tx -> {
32                     Result res =
33                         tx.run(
34                             """
35                             CALL gds.graph.list('paperCitations')
36                             YIELD nodeCount, relationshipCount
37                             RETURN nodeCount, relationshipCount
38                             """
39                         );
40                     return res.hasNext() ? res.single() : null;
41                 });
42
43         if (stats == null || stats.get("relationshipCount").asLong() == 0) {
44             log.warn("Graph projection has 0 relationships. Betweenness cannot be calculated
45                     .");
46             return Collections.emptyList();
47         }
48     }
49 }

```

```

46 // 3. Execute Algorithm with Pagination
47 List<Record> records =
48     session.executeRead(
49         tx -> {
50             Result res =
51                 tx.run(
52                     """
53                     CALL gds.betweenness.stream('paperCitations', {
54                         samplingSize: 1000,
55                         samplingSeed: 42
56                     })
57                     YIELD nodeId, score
58                     WITH nodeId, score
59                     ORDER BY score DESC
60                     SKIP $skip
61                     LIMIT $limit
62                     RETURN gds.util.asNode(nodeId).title AS title,
63                         round(score, 4) AS betweenness
64                     """,
65                     Map.of("skip", skip, "limit", limit));
66                 return res.list();
67             });
68
69     return records.stream().map(BetweennessEntry::new).collect(Collectors.toList());
70 } catch (Exception e) {
71     log.error("Betweenness calculation failed", e);
72     return Collections.emptyList();
73 }
74 }

```

This method will return a list of `BetweennessEntry`, composed by title of paper and betweenness centrality computed.

Chapter 7

Design Considerations

7.1 Indexes

7.1.1 Full-Text Search Implementation (Smart Search)

To provide users with a "Smart Search" capability, the system moves beyond exact-match equality filters and implements a Full-Text Search strategy. This allows users to find papers by querying keywords that may appear in either the title or the abstract, with results ranked by relevance.

1. Indexing Logic and Weights

Standard B-Tree indexes are inefficient for token-based searching inside string fields. Therefore, a specialized `text` index is utilized. The indexing strategy involves two key configuration parameters:

1. **Compound Indexing:** The index covers both the `title` and `abstract` fields, allowing a single search query to scan both distinct parts of the document simultaneously.
2. **Relevance Weighting:** Not all matches are equal. A keyword appearing in the *Title* is considered more significant than the same keyword appearing in the *Abstract*. To reflect this, distinct weights are assigned:
 - **Title:** Weight 10 (High Priority)
 - **Abstract:** Weight 3 (Standard Priority)

When MongoDB executes a search, it computes a *textScore* based on these weights. Documents matching the title will appear higher in the result list than those matching only the abstract.

2. Index Creation (Shell)

The following command creates the named index `textPaperIndex` on the `papers` collection, applying the weighting logic described above.

Listing 7.1: Creation of Weighted Text Index

```
1 db.papers.createIndex(  
2   {  
3     "title": "text",  
4     "abstract": "text"  
5   },  
6   {  
7     "weights": {  
8       "title": 10,  
9       "abstract": 3  
10    },  
11    "name": "textPaperIndex"  
12  }  
13 )
```

3. Java Implementation (Spring Data)

In the Repository layer, the search is exposed via a custom `@Query` method. The `$text` operator instructs MongoDB to use the pre-configured text index.

Listing 7.2: Repository method for Full-Text Search

```
1 Page<Paper> findByTextSearch(String keyword, Pageable pageable);
```

Note that `Pageable` is included in the method signature to support pagination of search results, which is essential for frontend performance when a keyword matches thousands of documents.

7.1.2 Optimization of Prefix Search (Autocomplete)

To support efficient autocomplete functionality (e.g., typing "Mar" to find "Mario"), the system relies on standard single-field indexes. Unlike the Full-Text search described previously, prefix searches (using Regex anchors like `^Pattern`) perform optimally with standard B-Tree indexes because the data is stored in alphabetical order.

Authors Name Index

The `authors` collection represents the largest dataset in this context (> 148,000 documents). Without an index, finding an author by name requires a full collection scan (COLLSCAN), which is computationally expensive.

1. Index Definition A standard ascending index was created on the `name` field.

Listing 7.3: Index creation for Author names

```
1 db.authors.createIndex(  
2   { "name": 1 },  
3   { "name": "authorsIndex" }  
4 )
```

2. Java Repository Implementation The Spring Data repository exposes this efficient search via a derived query method. The `StartsWith` keyword automatically translates to the efficient MongoDB regex pattern `/^prefix/`.

Listing 7.4: Repository method for Author Autocomplete

```
1 Page<Author> findByNameStartsWith(String name, Pageable pageable);
```

3. Performance Analysis A comparative analysis was conducted using the query `{ name: /^Mario/ }`. The execution statistics reveal a drastic difference in efficiency:

- **Without Index (Brute Force):** MongoDB performed a COLLSCAN, physically reading **148,434 documents** to find just 35 matches. The execution time was **195ms**. As the dataset grows, this approach becomes unsustainable.
- **With Index (Optimized):** MongoDB utilized an IXSCAN. It jumped directly to the letter "M" in the B-Tree, scanning only **36 keys** to return the same 35 documents. The execution time dropped to **0ms**.

Conclusion: The index reduced the workload by approximately **99.9%**. While a Text Index is powerful for keyword search, the Standard Index is mathematically superior for prefix/autocomplete tasks because it maintains alphabetical sorting.

User Management Indexes (RegisteredUser & Admin)

The same optimization strategy was applied to the `registeredUsers` and `admin` collections. Although these datasets are currently smaller, the index guarantees scalability and instant response times for searches.

1. Index Definitions and Java Methods Indexes were created on `full_name` for users and `username` for admins.

Listing 7.5: Index definitions for User Management

```
1 // Registered Users: Full Name Index
2 db.registeredUsers.createIndex( { "full_name": 1 }, { "name": "usersIndex" } )
3
4 // Admin: Username Index
5 db.admins.createIndex( { "username": 1 }, { "name": "adminIndex" } )
```

Correspondingly, the Java repositories define the following methods to leverage these indexes:

Listing 7.6: Repository methods for User Management

```
1 // For Registered Users
2 List<RegisteredUser> findByFullNameStartingWith(String prefix);
3
4 // For Admins
5 List<Admin> findByUsernameStartingWith(String prefix);
```

2. Registered Users Analysis For the query { `full_name: /^Alfreda/` }:

- **No Index:** The system examined **100 documents** (Total Scan) to return 0 results.
- **With Index:** The system utilized the `usersIndex`, examining only **3 index keys** and **2 documents**.

3. Admin Analysis For the query { `username: /^drake/` }:

- **No Index:** The system scanned **10 documents** to find 1 match.
- **With Index:** The system utilized the `adminIndex`, examining only **2 index keys** and **1 document**.

Summary of Performance Gains

Table 7.1 summarizes the impact of indexing across the three collections, highlighting the massive reduction in the number of documents examined.

Collection	Query Type	Docs Examined (No Index)	Docs Examined (Index)	Improvement
Authors	Name Prefix	148,434	35	$\approx 99.9\%$
RegisteredUsers	Name Prefix	100	2	98.0%
Admin	Username Prefix	10	1	90.0%

Table 7.1: Comparison of Documents Examined with and without Indexes

7.1.3 Neo4j Author name index

In order to improve the performance for queries that involve the name field of Author nodes, in particular `shortestPath` query. To create the index, the following Cypher command must be executed:

```
1 CREATE INDEX author_name_index IF NOT EXISTS FOR (a:Author) ON (a.name)
```

Once implemented performance improve significantly, here is a table that shows operations performed in order to execute query: before showing results, it is necessary to adopt **PROFILE** keyword in order to receive these result. For a complete view of improvements it is reported the report of execution when index is not implemented: As can be seen, the nodes accessed are decidedly greater than version with index.

Operator	Id	Details	Est. Rows	Rows	DB Hits	Pipeline	Index Used
ProduceResults	0	a.name	1	1	0	-	-
Projection	1	cache[a.name] AS a.name	1	1	0	-	-
NodeIndexSeek	2	RANGE INDEX a:Author(name) WHERE name = \$autostring_0	1	1	2	Fused in Pipeline 0	author_name_index

Table 7.2: Neo4j Profile: Optimized Query (Index Seek)

Operator	Id	Details	Est. Rows	Rows	DB Hits	Pipeline
ProduceResults	0	a.name	7422	1	0	-
Projection	1	cache[a.name] AS a.name	7422	1	0	-
Filter	2	cache[a.name] = \$autostring_0	7422	1	296868	-
NodeByLabelScan	3	a:Author	148434	148434	148435	Fused in Pipeline 0

Table 7.3: Neo4j Profile: Unoptimized Query (Full Scan)

7.2 Sharding

While the current implementation relies on a MongoDB Replica Set to ensure High Availability and read scalability, the system architecture is designed to support horizontal scaling via Sharding should the dataset exceed the capacity of a single node (e.g., multi-terabyte scale).

7.2.1 Sharding Strategy

We have identified the **papers** collection as the primary candidate for sharding due to its cardinality and continuous growth.

- **Strategy: Hashed Sharding** on the `_id` field.
- **Rationale:**
 - **Write Distribution:** Scientific papers are ingested continuously. A Range-based sharding strategy on a field like **year** would result in a "Monotonically Increasing Key" problem, where all new writes target the single shard responsible for the current time range (Hot Shard). Hashing the `_id` ensures that writes are randomly and evenly distributed across all shards in the cluster.
 - **Balanced Storage:** This strategy guarantees that chunks are balanced across shards, optimizing storage utilization.

7.3 Usage of Virtual Machines

The production environment is orchestrated across three Virtual Machines (VMs) to simulate a distributed system. The allocation of services, defined in the `deploy_orchestrator.sh` script, is as follows:

Node	IP Address	Hosted Services
VM 1 (Primary)	10.1.1.80	• MongoDB Primary (mongo1): Handles writes. • Neo4j Enterprise: Graph Database. • Backend API: Spring Boot Application.
VM 2 (Worker)	10.1.1.78	MongoDB Secondary (mongo2)
VM 3 (Worker)	10.1.1.77	MongoDB Secondary (mongo3)

Table 7.4: Virtual Machine Service Map

This topology ensures that the Database layer maintains High Availability. If VM 1 fails, an election occurs between VM 2 and VM 3 to choose a new MongoDB Primary, preserving data access.

7.4 CAP Theorem reflections

7.4.1 Neo4j

This part of the system is AC (Availability + Consistency) because there is only 1 instance of Neo4j graphDB implemented on one node. Data retrieved from this one are always consistent and both read and write operation could be done without any restrictions. Lack of Partition Tolerance comes from the fact that in our system there's only one active instance of neo4j running, so when server (host) goes down, Neo4j services are unavailable.

7.4.2 MongoDB

In our implementation we set that Mongo DB can only perform read operation only on the primary, so users can always see the last version of the data, so this demonstrates that our system is always in a consistent state (C). We also set that MongoDB's write operations do not return OK until they are performed to the majority of servers, this guarantees that even if the primary crashes immediately after a write, the data is preserved on the secondaries, preventing scenarios where data is lost. We set that if a network partition occurs this happens:

1. The isolated Primary will step down because it cannot see a majority.
2. The remaining nodes needs time to hold an election to pick a new Primary. During this phase read and write operation are blocked.

So this demonstrates that our system is partition tolerant (P). After this 2 demonstrations we can conclude that our system is CP.

7.5 Management of the consistency between the two types of Databases

We first write data to MongoDB and immediately after to Neo4j. The consistency mechanism relies on Spring's @Transactional annotation combined with the MongoDB Replica Set capabilities. This is the process flow:

1. A create/update request enters the Service Layer.
2. The @Transactional annotation begins a MongoDB transaction.
3. The entity is saved to MongoDB.
4. The system attempts to write the corresponding nodes/relationships.
5. If Neo4j fails, a custom DAOException is thrown.
6. The Exception propagates up, causing the Spring Transaction Manager to roll back the MongoDB operation, ensuring atomicity.

Chapter 8

Testing and Validation

To ensure the reliability and robustness of Nexus Scholar, we employ a comprehensive testing strategy that spans Unit Testing, Integration Testing with containerized infrastructure, and Continuous Integration (CI).

8.1 Testing Stack

The project utilizes the following technologies:

- **JUnit 5:** The primary testing framework.
- **Testcontainers:** A Java library that supports JUnit tests by providing lightweight, throwaway instances of common databases running in Docker containers. This allows us to test against **real** MongoDB and Neo4j instances rather than mocks.
- **Mockito:** Used in unit tests to isolate business logic by mocking dependencies.
- **JaCoCo:** Used to measure code coverage.
- **Spotless:** Enforces Google Java Format code style.

8.2 Test Categories

8.2.1 Unit Tests

Located in `src/test/java`, unit tests verify the correctness of individual classes in isolation.

- **Scope:** Utility classes (e.g., `PageRankEntryTest`), DTO serialization, and Service layer logic (e.g., `AuthServiceImplTest` verifying password matching).
- **Performance:** These tests execute quickly as they do not require loading the full Spring Application Context.

8.2.2 Integration Tests

Integration tests verify the interaction between the application and the persistence layer.

- **Infrastructure:** The `TestcontainersConfiguration` class automatically spins up a `mongo:8.2.3` container and a `neo4j:2025.11.2` container (with GDS plugins) before tests run.
- **Verification:**
 - **MongoDB:** Verifies custom queries, Full-Text Search, and complex Aggregation Pipelines (e.g., `PaperAnalysisDAOImplTest`).
 - **Neo4j:** Verifies Cypher query correctness, node creation, and graph algorithm execution (e.g., `GraphDAOIntegrationTest`).

8.3 Continuous Integration (CI)

A GitHub Actions workflow (`backend-ci.yml`) is triggered on every push to the repository. It performs the following quality checks:

1. **Format Check:** Runs `mvn spotless:check` to ensure code style compliance.
2. **Build & Test:** Compiles the code and runs the full test suite (`mvn clean verify`).
3. **Static Analysis:** Uploads coverage reports to Codecov and performs static analysis via SonarCloud to detect bugs and code smells.

Chapter 9

Conclusions

9.1 Future works

This application implements RESTful API that is used for exploring world of researchers, although it implements a lot of functionalities, expansions could be done in several aspects. For example the front-end side could be effectively implemented, starting from mockups reported above in this documentation. Other useful aspects could be the integration of other neo4j algorithms, that gives the user useful information about the graph, offered by Graph Data Science (for example) the same applies to the MongoDB side which could also be extended considering more Aggregation pipeline that will work with papers. Our application doesn't contain all papers for practical reasons, but with more powerful infrastructures, more data could be imported in data-pipeline phase and maybe including more sources, even if including different kinds of format.